Université Hassan 1er
Faculté des Sciences et Techniques
-Settat-

Projet de Fin d'Etude

En vue de l'obtention du diplôme

MASTER SCIENCES ET TECHNIQUES

Spécialité : Réseaux et Systèmes Informatiques (RSI)

Présenté par

AFAKHAR IKRAM

Conception et développement de Segula Homologation 360 : une application web intelligente pour la gestion des essais d'homologation

Sous la direction de :

Pr. Imane Lmati, Professeur à la FST de Settat

Mme. Siham Chahm , Responsable Projet Homologation – SEGULA Technologies

Mr. Boukabou Ismail, Ingénieur Homologation – SEGULA Technologies

Soutenu le 25 juin 2026, devant le jury :

Mr. El Kafhali Said	Professeur à la FST de Settat	Examinateur
Mme. Amina Ouatiq	Professeur à la FST de Settat	Examinatrice
Mme. Imane Lmati	Professeur à la FST de Settat	Encadrante

Année Universitaire : 2025 - 2026

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

اقْرَأْ بِاسْمِ رَبِّكَ الَّذِي خَلَقَ

خَلَقَ الْإِنسَانَ مِنْ عَلَقٍ

اقْرَأْ وَرَبُّكَ الْأَكْرَمُ

الَّذِي عَلَّمَ بِالْقَلَمِ

عَلَّمَ الْإِنسَانَ مَا لَمْ يَعْلَمْ

﴿سورة العلق: 1-5﴾

Dédicace

Je tiens à exprimer ma profonde gratitude :

À mes parents, pour leur amour, leurs sacrifices et leur soutien constant, sans lesquels ce parcours n'aurait pas été possible.

À mes professeurs et encadrants, pour leur accompagnement, leur exigence bienveillante et leurs précieux conseils qui ont transformé mes doutes en détermination.

À ma famille et mes amis, pour leur compréhension, leur soutien et leur présence rassurante dans les moments difficiles.

À tous ceux qui ont cru en moi et m'ont encouragé, merci d'avoir été la source de ma motivation.

Ce travail est le fruit de votre confiance et de votre accompagnement.

Merci du fond du cœur

Remerciements

Avant toute chose, je rends grâce à Allah pour la force, la patience et la persévérance qu'Il m'a accordées, me permettant de mener ce projet à son terme avec succès.

Je souhaite exprimer ma profonde gratitude à toutes les personnes ayant contribué au bon déroulement et à la réussite de mon stage au sein de Segula Technologies.

Je remercie sincèrement mon encadrant, **Mr. Ismail Boukabou**, ma manager, **Mme. Siham Chahm**, ainsi que mon encadrante pédagogique, **Mme. Lmati Imane**, pour leur accompagnement, leurs conseils avisés et leur soutien constant tout au long de ce projet. Leur expertise et leurs encouragements ont été essentiels à la réussite de cette expérience professionnelle.

Je remercie également **Mme. Hanae Nehhale**, responsable des Ressources Humaines, ainsi qu'à toute **l'équipe des Ressources Humaines**, pour leur aide précieuse dans les démarches administratives et pour avoir facilité mon intégration au sein de Segula Technologies.

Ma reconnaissance s'adresse aussi aux **membres du jury** pour l'évaluation de mon travail et leurs remarques constructives, qui ont permis d'enrichir ce projet.

Je remercie l'ensemble du corps professoral de **la Faculté des Sciences et Techniques de Settat**, les intervenants de la formation **RSI**, ainsi que le chef de filière **RSI**, **Mme. Sara Arezki**, pour leur encadrement tout au long de mon parcours académique.

Enfin, j'adresse une pensée particulière à **ma famille** et à **mes amis** pour leur soutien indéfectible, leur confiance et leur encouragement constant.

Je remercie toutes les personnes, de près ou de loin, ayant contribué à la réussite de ce stage, faisant de cette expérience une étape marquante de mon parcours professionnel.

Résumé

Le secteur automobile est soumis à des exigences réglementaires strictes, notamment en matière d'homologation des véhicules. Ce projet de fin d'études, réalisé au sein de Segula Technologies, a pour objectif de concevoir et développer une application web intelligente permettant de centraliser, automatiser et optimiser la gestion des règlements d'homologation des véhicules.

La solution proposée repose sur une architecture 3-tiers composée d'un frontend React avec TypeScript, d'un backend Spring Boot et d'une base de données PostgreSQL, enrichie par une couche d'intelligence artificielle basée sur le modèle Llama 3 via l'API Groq. Le premier module permet la gestion complète des règlements d'homologation, incluant les conditions d'essai, les paramètres de mesure, les critères de conformité, et propose pour chaque règlement une présentation interactive sous forme de slides pédagogiques détaillant étape par étape ses protocoles d'essais. Un assistant IA conversationnel, reposant sur une architecture RAG (Retrieval-Augmented Generation), offre une assistance réglementaire en langage naturel. Le module de visualisation 3D, développé en Three.js, propose une représentation interactive des véhicules et permet de filtrer les règlements par pièce et par type de motorisation. Enfin, la plateforme intègre un système de gestion documentaire avec extraction intelligente de contenus PDF.

Le projet adopte une approche DevOps avec un pipeline d'intégration continue via GitHub Actions et un déploiement conteneurisé via Docker Compose, garantissant la portabilité et la fiabilité de la solution. L'application intègre un système d'authentification sécurisé basé sur JWT avec gestion des rôles. L'orchestration via Kubernetes est envisagée comme perspective d'évolution vers un environnement cloud en production.

Cette plateforme permet d'améliorer significativement la productivité, la conformité réglementaire et la collaboration entre les différents acteurs du processus d'homologation.

Abstract

The automotive industry operates under stringent regulatory frameworks, particularly regarding vehicle type-approval. This final-year project, conducted at Segula Technologies, presents the design and development of an AI-powered web platform for centralizing and streamlining the management of vehicle homologation regulations.

Built on a three-tier architecture comprising a React/TypeScript frontend, a Spring Boot backend, and a PostgreSQL database, the platform leverages the Llama 3 model through the Groq API to deliver intelligent features. The core module handles the full lifecycle of homologation regulations, covering test conditions, measurement parameters, compliance criteria, and features an interactive pedagogical slide presentation for each regulation to detail its step-by-step test protocols. A conversational AI assistant, built on a Retrieval-Augmented Generation (RAG) pipeline, provides regulatory guidance in natural language. The 3D visualization module, developed with Three.js, offers an interactive vehicle representation and enables regulation filtering by part and powertrain type. Finally, the platform includes a document management system featuring AI-driven PDF content extraction.

The project follows a DevOps methodology with a continuous integration pipeline powered by GitHub Actions and containerized deployment through Docker Compose, ensuring portability and reliability. Secure authentication is enforced via JSON Web Tokens with role-based access control. Future work includes Kubernetes orchestration for cloud-native production deployment.

The resulting platform significantly improves productivity, regulatory compliance, and cross-functional collaboration throughout the homologation process.

Table de matière

Liste des figures 8
Liste des abréviations 11
Introduction générale 12
Chapitre 1 14
1. Introduction 14
2. Présentation de l'organisme d'accueil 15
3. Organisation du lieu de stage et du département 21
4. Conclusion 23
Chapitre 2 24
1. Introduction 24
2. Contexte et problématique 25
3. Solution proposée 26
Fonctionnalités principales 26
4. Analyse détaillée des besoins 28
5. Contraintes et enjeux 29
6. Planification du projet 30
7. Conclusion 32
Chapitre 3 33
1. Introduction 33
2. Architecture générale du système 34
3. Choix techniques 35
4. Modélisation UML 37
5. Gestion documentaire et collecte réglementaire 55
6. Sécurité et gestion des accès 55
7. Extensibilité et évolutivité 55
8. Conclusion 56
Chapitre 4 57
1. Introduction 57
2. Outils et technologies 58
3. Outils bureautiques et ressources externes 66
4. Présentation des interfaces utilisateur 67
5. Intégration et déploiement 79
6. Conclusion 82
Chapitre 5 83
1. Introduction 83
2. Résultats obtenus 84
3. Limites de la solution 87
4. Perspectives d'évolution 88
5. Analyse critique 91
6. Conclusion 92
Conclusion générales 93
Webographie Et Bibliographie 94

Liste des figures

Figure 1 : Logo de SEGULA Technologies 15
Figure 2 : Implantation de SEGULA Technologies à l'échelle internationale 16
Figure 3 : Répartition du chiffre d'affaires de SEGULA Technologies par secteurs 17
Figure 4 : Répartition géographique du chiffre d'affaires de SEGULA Technologies ... 17
Figure 5 : Les secteurs d'activités de SEGULA Groupe 18
Figure 6 : Les clients du groupe SEGULA 19
Figure 7 : Organigramme de SEGULA Technologies Maroc 21
Figure 8 : Architecture de la solution 27
Figure 9 : Approche Devops 28
Figure 10 : Diagramme de Gantt 31
Figure 11 : Logo UML 38
Figure 12 : Diagramme de cas d'utilisation 39
Figure 13 : Diagramme de classes 42
Figure 14 : Diagramme de séquence - Connexion à l'interface 43
Figure 15 : Diagramme de séquence - Mot de passe oublié 44
Figure 16 : Diagramme de séquence - Interface Dashboard 44
Figure 17 : Diagramme de séquence - Affichage des règlements 45
Figure 18 : Diagramme de séquence - Consultation d'un règlement 45
Figure 19 : Diagramme de séquence - Création / Modification manuelle 46
Figure 20 : Diagramme de séquence - Extraction PDF par IA 46
Figure 21 : Diagramme de séquence - Visualisation 3D 47
Figure 22 : Diagramme de séquence - Assistant IA 47
Figure 23 : Diagramme de séquence - Interface administration : Gestion des utilisateurs 48
Figure 24 : Diagramme de séquence - Interface des documents réglementaires 49
Figure 25 : Diagramme de séquence - Déconnexion et thème 50
Figure 26 : Diagramme d'activité 51
Figure 27 : Diagramme d'architecture technique 54
Figure 28 : Logo de Java 58
Figure 29 : Logo de TypeScript 59
Figure 30 : Logo de JavaScript 59
Figure 31 : Logo de PostgreSQL 59
Figure 32 : Logo de Python 60
Figure 33 : Logo de React 60
Figure 34 : Logo de Vite 60
Figure 35 : Logo de Tailwind CSS 60
Figure 36 : Logo de ThreeJS 61
Figure 37 : Logo de Axios 61
Figure 38 : Logo de React Router 61
Figure 39 : Logo de SpringBoot 62
Figure 40 : Logo de Flyway 62
Figure 41 : Logo de Maven 63
Figure 42 : Logo de PDFBox 63
Figure 43 : Logo de Docker 63
Figure 44 : Logo de Nginx 64
Figure 45 : Logo de Kubernetes 64

Figure 46 : Logo de Git 64
Figure 47 : Logo de github 64
Figure 48 : Logo de Prometheus 65
Figure 49 : Logo de Grafana 65
Figure 50 : Interface de connexion 68
Figure 51 : Interface de Tableau de bord 69
Figure 52 : Interface de liste des règlements 69
Figure 53 : Interface de détail d'un règlement 71
Figure 54 : Interface - Formulaire de création / modification d'un règlement 72
Figure 55 : Interface d'extraction automatique depuis un PDF 72
Figure 56 : Interface - Assistant IA 75
Figure 57 : Pipeline RAG 76
Figure 58 : Interface - Visualisation 3D du véhicule 78
Figure 59 : Interface de gestion des utilisateurs 78
Figure 60 : Interface de catalogue de documents réglementaires 79
Figure 61 : Architecture technique de la solution 79
Figure 62 : Pipeline CI avec GitHub Actions 80

Liste des tableaux

Table 1 : Fiche signalétique du groupe SEGULA 16
Table 2 : Fiche signalétique de SEGULA Technologies Maroc 20
Table 3 : Validation des besoins identifiés 86

Liste des abréviations

Abréviation	Définition
API	Application Programming Interface (Interface de programmation applicative)
CI/CD	Continuous Integration / Continuous Deployment (Intégration et déploiement continus)
CNRV	Centre National de Réception des Véhicules
CRUD	Create, Read, Update, Delete (Créer, Lire, Mettre à jour, Supprimer)
DevOps	Development and Operations (Développement et Opérations)
ERP	Enterprise Resource Planning (Progiciel de Gestion Intégré)
GED	Gestion Électronique des Documents
GLB	Format de fichier 3D binaire (GL Transmission Format Binary)
IA	Intelligence Artificielle
JWT	JSON Web Token (Jeton d'authentification web)
KPI	Key Performance Indicator (Indicateur Clé de Performance)
LLM	Large Language Model (Grand Modèle de Langage)
PDF	Portable Document Format
PostgreSQL	Post Structured Query Language (Système de gestion de base de données relationnelle)
RAG	Retrieval-Augmented Generation (Génération augmentée par récupération)
REST	Representational State Transfer (Style d'architecture web)
RGPD	Règlement Général sur la Protection des Données
RHN	Réglementation, Homologation, Normalisation
RSI	Réseaux et Systèmes Informatiques
SaaS	Software as a Service (Logiciel en tant que service)
SQL	Structured Query Language (Langage de requête structuré)
UML	Unified Modeling Language (Langage de modélisation unifié)
UNECE	United Nations Economic Commission for Europe (Commission économique des Nations Unies pour l'Europe)
URL	Uniform Resource Locator (Adresse web)
UTAC	Union Technique de l'Automobile, du motocycle et du Cycle
VHM	Vehicle Homologation Management (Gestion de l'homologation des véhicules)
WebSocket	Protocole de communication bidirectionnel en temps réel

Introduction générale

Dans un contexte marqué par la transformation numérique, l'industrie automobile et les secteurs connexes sont confrontés à des exigences croissantes en matière d'innovation, de conformité réglementaire et d'optimisation des processus métiers. L'évolution rapide des normes, combinée à la complexité des systèmes techniques, rend l'accès et l'interprétation des réglementations particulièrement critiques pour garantir la fiabilité et la conformité des véhicules.

Dans ce cadre, Segula Technologies, acteur international de référence en ingénierie et en conseil, accompagne ses clients dans la conception, la validation et la mise en conformité de solutions innovantes. Cependant, la consultation et la structuration des normes d'homologation demeurent encore largement manuelles, dispersées dans des documents textuels denses, ce qui engendre des pertes de temps, un risque accru d'erreurs d'interprétation et un manque d'accessibilité globale à l'information technique.

C'est dans cette optique que s'inscrit le projet de fin d'études intitulé « Segula Homologation 360 », réalisé au sein de Segula Technologies. Ce projet a pour objectif de concevoir et de développer une plateforme web permettant de centraliser l'ensemble des règlements d'homologation dans un référentiel unique et d'en faciliter l'accès. Plutôt que de se limiter à la simple gestion de processus, la solution proposée vise à révolutionner la manière dont les ingénieurs interagissent avec la donnée réglementaire à travers :

- La centralisation et la structuration intelligente des informations et documents techniques.
- L'intégration d'une assistance automatisée pour faciliter la recherche, l'extraction et l'analyse des données.
- L'utilisation d'interfaces visuelles avancées permettant de relier intuitivement les normes aux composants physiques étudiés.
- La mise en place de parcours interactifs conçus pour guider les utilisateurs dans l'appréhension et le déroulement des processus de conformité.

La solution développée repose sur une architecture 3-tier cloud-native, composée d'une couche de présentation dynamique assurée par des technologies web modernes intégrant le rendu 3D, d'une couche métier exposée via des API REST reposant sur des frameworks robustes, ainsi que d'une couche de données basée sur une base relationnelle. L'orchestration des services, pensée pour être évolutive via des outils d'orchestration de conteneurs, garantit la scalabilité, la haute disponibilité et une gestion centralisée des déploiements. Cette architecture assure robustesse et extensibilité, soutenant efficacement l'intégration poussée de l'intelligence artificielle au cœur de l'application.

Le présent rapport est structuré en cinq chapitres :

Chapitre 1 : Présentation de l'organisme d'accueil Ce chapitre présente Segula Technologies, ses domaines d'activité ainsi que son positionnement dans le secteur de l'ingénierie.

Chapitre 2 : Étude préalable et analyse des besoins Il expose le contexte du projet, les problématiques liées à la dispersion de l'information réglementaire, ainsi que les besoins fonctionnels et non fonctionnels.

Chapitre 3 : Conception de la solution Ce chapitre décrit l'architecture du système, les choix techniques ainsi que la modélisation à l'aide de diagrammes UML.

Chapitre 4 : Réalisation et implémentation Il présente les différentes phases de développement, les technologies globales utilisées ainsi que des illustrations des fonctionnalités mises en œuvre.

Chapitre 5 : Résultats, limites et perspectives Ce chapitre propose une évaluation de la solution, met en évidence ses limites et présente les perspectives d'évolution vers un déploiement continu.

Ainsi, à travers la centralisation intelligente de la documentation technique, le projet « Segula Homologation 360 » contribue à faciliter l'accès à l'information, à renforcer la fiabilité des analyses réglementaires et à répondre aux enjeux actuels de conformité et de compétitivité des ingénieurs homologation.

Chapitre 1

Présentation de l'organisme d'accueil

1. Introduction

Ce chapitre offre une présentation détaillée de SEGULA Technologies, en retraçant son parcours historique, son chiffre d'affaires, ses principaux secteurs d'activité ainsi que sa clientèle, avec une mention particulière pour le groupe Stellantis. Il aborde également le cycle de vie du véhicule au sein de l'usine, ses différentes typologies, le processus de fabrication, ainsi que les outils de conception utilisés.

2. Présentation de l'organisme d'accueil

2.1. Groupe SEGULA

Le groupe SEGULA Technologies est présent dans plus de 30 pays, avec environ 140 implantations à travers le monde. Il privilégie une relation de proximité avec ses clients grâce aux compétences de ses 12 000 à 15 000 collaborateurs.

Acteur majeur de l'ingénierie, plaçant l'innovation au cœur de sa stratégie, SEGULA Technologies mène des projets d'envergure couvrant l'ensemble des phases, depuis les études jusqu'à l'industrialisation et la production.

Figure 1 : Logo de SEGULA Technologies

Fondé en 1985, SEGULA Technologies est un groupe d'ingénierie et de conseil en innovation dirigé par un directoire présidé par Franck Ghrenassia. Son siège social est situé à Nanterre. Initialement centré sur l'industrie automobile qui demeure son principal secteur d'activité le groupe s'est progressivement diversifié vers d'autres domaines tels que l'aéronautique, l'énergie, le ferroviaire et le naval, notamment à travers des acquisitions stratégiques.

SEGULA Technologies accompagne ses clients tout au long de leurs projets techniques, depuis les phases d'études, de management et de conception jusqu'à la fiabilisation, l'industrialisation et la production, en proposant des solutions sous forme de conseil ou de mise à disposition de collaborateurs spécialisés.

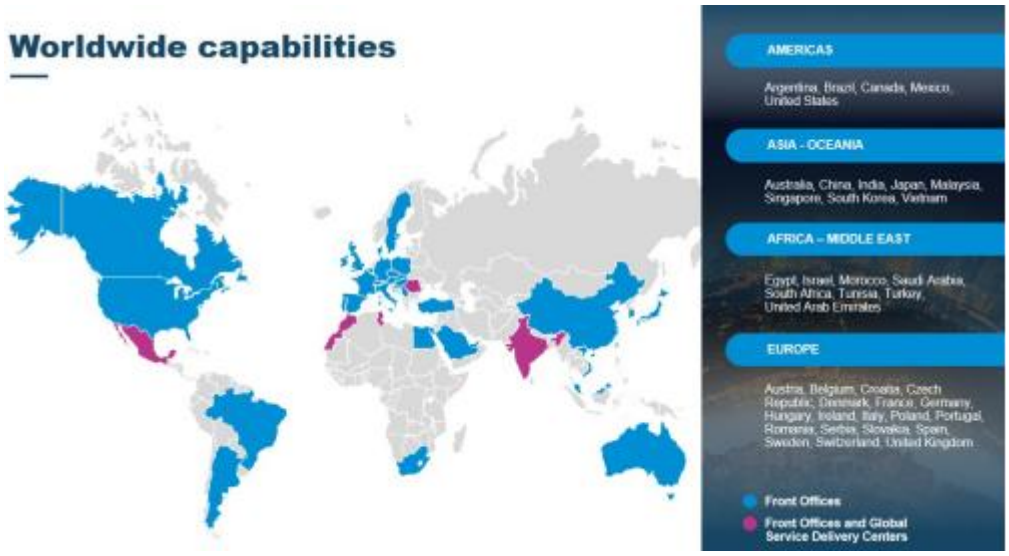


Figure 2 : Implantation de SEGULA Technologies à l'échelle internationale

2.2. Fiche signalétique de l'entreprise

Le tableau ci-dessous présente les principales informations relatives au groupe SEGULA Technologies.

Raison sociale	SEGULA Technologies
Forme juridique	Société par Actions Simplifiée (SAS)
Activité	Groupe d'ingénierie et de conseil en innovation
Date de création	1985, France
Implantations	140 implantations dans plus de 30 pays
Adresse du siège	19 rue Dares, 92000 Nanterre
Chiffre d'affaires	Plus de 538 millions d'euros (2023)[6]
Site Web	www.segulatechnologies.com

Table 1 : Fiche signalétique du groupe SEGULA

2.3. Mission et chiffre d'affaires

SEGULA Technologies figure parmi les principaux groupes européens de conseil et d'ingénierie en innovation technologique. Il intervient sur l'ensemble du cycle de vie des projets, depuis leur définition (veille technologique, études de faisabilité, élaboration de stratégies, etc.) jusqu'à leur concrétisation (conception, mise en œuvre et validation de solutions).

Quelques chiffres clés :

- Plus de 12 000 collaborateurs et plus de 300 clients industriels majeurs
- 2 250 recrutements en 2008, dont 1 500 en France et 750 à l'international
- 538 millions d'euros de chiffre d'affaires
- 20 % du chiffre d'affaires réalisé à l'international



Figure 3 : Répartition du chiffre d'affaires de SEGULA Technologies par secteurs

L'activité de SEGULA Technologies s'articule autour de deux axes principaux :

- Conseil en technologie et en recherche & développement
- Conseil en organisation et en systèmes d'information

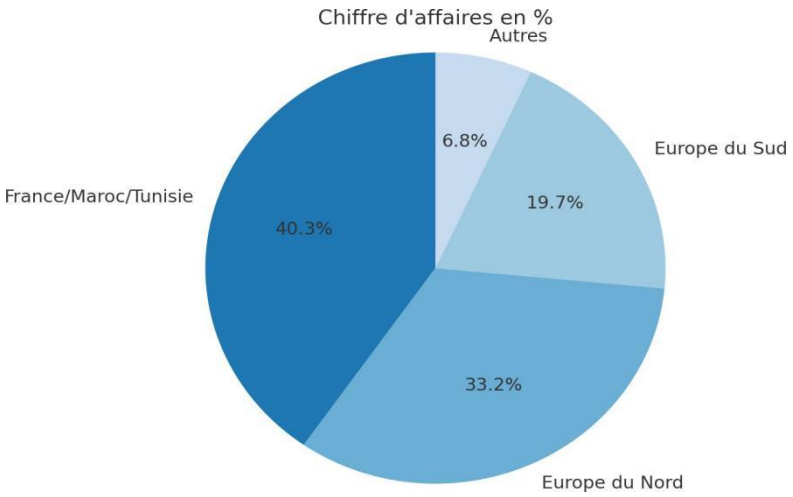


Figure 4 : Répartition géographique du chiffre d'affaires de SEGULA Technologies

2.4. Les secteurs d'activité de SEGULA Groupe

SEGULA Technologies est un groupe d'ingénierie international qui accompagne la compétitivité de nombreux secteurs industriels. Ses activités couvrent notamment les domaines de l'automobile, de l'aéronautique, de l'énergie, du ferroviaire, du naval, de la pharmacie ainsi que du pétrole et du gaz.



Figure 5 : Les secteurs d'activités de SEGULA Groupe

Automobile & véhicules industriels
Présent dans de nombreuses grandes villes à travers le monde de Mexico à Shanghai, en passant par Détroit, Paris, Lyon, Casablanca, Bucarest, Bratislava, Pékin ou Hong Kong SEGULA Technologies rassemble des milliers d'experts du secteur automobile. Le groupe conçoit les véhicules du futur et développe des sites industriels plus flexibles et performants, en proposant des solutions adaptées aux besoins spécifiques de chaque client.

Aéronautique, spatial et défense
La filiale SIMRA propose des solutions intégrées couvrant l'ensemble du cycle de vie des aéronefs, de la conception à la maintenance. Les équipes interviennent sur différents types d'appareils : avions civils et militaires, hélicoptères et satellites.

Énergie
SEGULA Technologies intervient dans les secteurs de l'énergie nucléaire, thermique et renouvelable. Il accompagne ses clients sur l'ensemble du cycle de vie des projets : conception, construction, mise en service, exploitation, maintenance et démantèlement.

Ferroviaire
Le groupe contribue au développement de projets ferroviaires à l'échelle mondiale, depuis la conception jusqu'à la maintenance des systèmes et infrastructures.

Naval
SEGULA Technologies intervient dans la conception, l'industrialisation et le maintien

en condition opérationnelle de navires civils et militaires, ainsi que dans les infrastructures portuaires et offshore.

Pétrochimie
Le groupe accompagne les acteurs du raffinage et de la chimie tout au long du cycle de vie de leurs installations, des études de faisabilité à la supervision des travaux.

Pharmacie
Depuis plus de dix ans, SEGULA Technologies soutient les acteurs des sciences de la vie en proposant :

- Des solutions conformes aux exigences réglementaires,
- Des stratégies d'optimisation des performances industrielles,
- Des études d'ingénierie pour le développement des capacités de production.

2.5. Clients de SEGULA

SEGULA Technologies est un acteur majeur de l'ingénierie au service de grands groupes industriels. Dans le secteur automobile, Stellantis figure parmi ses principaux partenaires. La figure ci-dessous présente un aperçu des principaux clients du groupe SEGULA Technologies :



Figure 6 : Les clients du groupe SEGULA

2.6. SEGULA Technologies Maroc

2.6.1. Présentation

SEGULA Technologies Maroc intervient principalement dans le secteur automobile à travers ses bureaux d'études situés à Casablanca, Agadir et Tanger, ainsi que dans le secteur aéronautique via sa filiale SIMRA.

Dans le domaine automobile, l'entreprise réalise de nombreux projets en mode forfaitaire, accompagnant ses clients dans la conception et l'industrialisation de nouveaux produits ainsi que dans la création de nouvelles usines.

Dans le secteur aéronautique, SIMRA collabore avec des industriels marocains et internationaux. Le groupe a récemment renforcé son expertise dans les métiers de la tôlerie et du traitement de surface.

2.6.2. Fiche signalétique de SEGULA Technologies Maroc

Le tableau ci-dessous présente les principales informations relatives à l'entreprise.

Raison sociale	SEGULA Technologies Maroc
Forme juridique	Société Anonyme
Activité	Groupe d'ingénierie
Date de création	2018
Domaines d'expertise	Plan de forme Roulage (Kénitra), optimisation des coûts RHN, Conception
Implantations	- Casablanca Nearshore Park, 1100 Bd. Al Qods, Shore 26, Sidi Maarouf, 20270 Casablanca - Park View Square, La Baie des Palmiers, immeuble 56 Founty, 80010 Agadir - Bureau C1-B, 3e étage, Centre d'Affaires ZF Business Center, lot 40-A, Zone Franche d'Exportation, Tanger - Zone Technopole Aéroport Mohamed V (site de la filiale SIMRA)
Site web	https://morocco.segulatechnologies.com

Table 2 : Fiche signalétique de SEGULA Technologies Maroc

2.6.3. Organigramme de la société

Une entreprise est une organisation hiérarchique regroupant différentes activités. L'organigramme de SEGULA Technologies Maroc, où s'est déroulé le stage de fin d'études, est présenté ci-dessous :

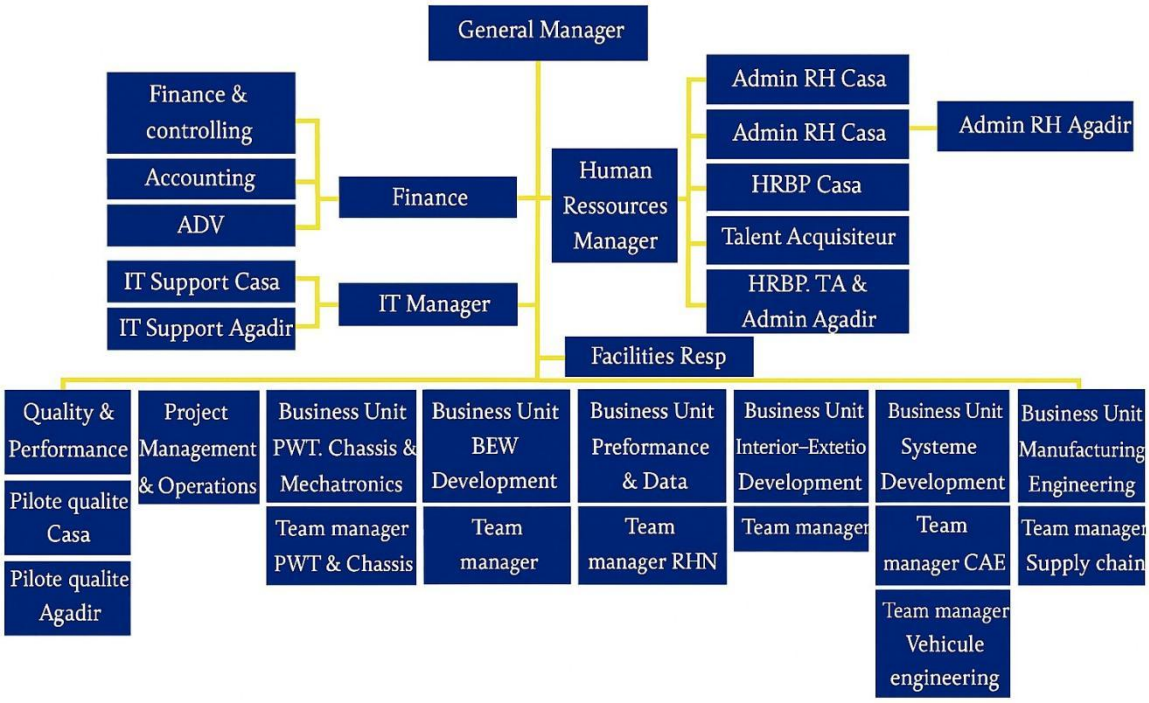


Figure 7 : Organigramme de SEGULA Technologies Maroc

3. Organisation du lieu de stage et du département

3.1. Description du lieu de travail

Le stage de fin d'études s'est déroulé au sein de SEGULA Technologies Maroc, basé à Casablanca.

L'environnement de travail est organisé en espace ouvert (open space), favorisant la communication, la collaboration et l'échange entre les différentes équipes techniques. Cette organisation permet une meilleure coordination entre les services et un suivi efficace des projets, notamment dans le domaine de l'ingénierie automobile.

Le cadre professionnel est dynamique et orienté vers la performance, avec un fort accent sur la qualité, le respect des délais et la conformité aux exigences industrielles.

3.2. Présentation du service RHN

Le service RHN (Réglementation, Homologation, Normalisation) est un département clé au sein de SEGULA Technologies Maroc.

Sa mission principale est de garantir l'homologation des véhicules du groupe Stellantis, en assurant leur conformité aux réglementations en vigueur afin de permettre leur mise en circulation sur les marchés internationaux.

Les principales activités du service RHN sont :

- Analyse et anticipation des impacts des évolutions réglementaires
- Gestion et validation des fichiers techniques fournis par les fournisseurs
- Préparation des dossiers techniques d'homologation (RCE)
- Suivi des essais et gestion des retours des organismes officiels (UTAC, CNRV)
- Élaboration de rapports de synthèse pour le suivi des projets
- Coordination avec le client Stellantis et les équipes internes

Ainsi, le service RHN joue un rôle essentiel dans la validation finale des véhicules avant leur commercialisation.

3.3. Rôle de l'homologation dans le processus industriel

L'homologation est une étape indispensable permettant de certifier qu'un véhicule respecte les exigences techniques et réglementaires en matière de sécurité et d'environnement.

Elle repose sur :

- La réalisation d'essais techniques
- La vérification de la conformité réglementaire
- L'obtention d'un certificat officiel délivré par des organismes compétents

Ce processus implique plusieurs acteurs, notamment :

- L'UTAC pour les essais techniques
- Le CNRV pour la certification

3.4. Présentation de l'équipe VHM

Au sein du service RHN, le stage s'est déroulé dans l'équipe VHM (Vehicle Homologation Management).

Cette équipe joue un rôle central dans le pilotage des activités d'homologation. Elle assure la coordination entre les différents périmètres impliqués dans le processus.

Ses principales missions sont :

- Pilotage et suivi de l'avancement des activités d'homologation
- Coordination entre les équipes Environnement, Sécurité et Homologation
- Planification des tâches et gestion des priorités
- Interface entre les équipes techniques et les parties prenantes
- Consolidation des données et reporting global

L'équipe VHM constitue ainsi un point de liaison stratégique, garantissant la cohérence et la synchronisation des travaux entre les différents acteurs du projet.

Elle contribue à améliorer :

- La visibilité sur l'avancement des projets
- La qualité des livrables
- L'efficacité globale du processus d'homologation

4. Conclusion

En résumé, ce chapitre a permis de présenter l'organisme d'accueil, son environnement de travail ainsi que le service RHN et l'équipe VHM dans lesquels s'est déroulé le stage.

Cette analyse a offert une compréhension claire du fonctionnement de l'entreprise et du rôle stratégique de l'homologation dans l'industrie automobile. Elle constitue une base essentielle pour aborder le chapitre suivant consacré à l'étude du projet et à l'analyse des besoins.

Chapitre 2

Étude préalable et analyse des besoins

1. Introduction

Ce chapitre présente le cadre général du projet « Segula Homologation 360 ». Il met en évidence les problématiques liées à la gestion et à la consultation des réglementations d'homologation, explique le rôle des règlements internationaux, décrit la solution proposée et formalise l'ensemble des besoins fonctionnels et non fonctionnels.

L'objectif est d'établir une base solide pour la conception d'une plateforme innovante permettant de centraliser, consulter et explorer les réglementations d'homologation automobile, en adéquation avec les exigences du secteur automobile et les attentes des utilisateurs métiers.

2. Contexte et problématique

2.1. Rôle des règlements d'homologation

L'homologation automobile constitue un processus réglementaire fondamental qui vise à garantir qu'un véhicule respecte l'ensemble des exigences techniques, de sécurité, environnementales et de qualité avant sa mise sur le marché. Ce processus est encadré par des règlements internationaux, régionaux et nationaux, notamment les règlements de la Commission économique des Nations Unies pour l'Europe (UNECE), les directives européennes, ainsi que certaines réglementations spécifiques à des pays ou zones géographiques.

Ces règlements d'homologation constituent un référentiel normatif structurant pour l'ensemble du cycle de validation des véhicules. Ils définissent de manière précise :

- Les types d'essais à réaliser (sécurité passive, sécurité active, émissions polluantes, compatibilité électromagnétique, performance énergétique, etc.).
- Les conditions expérimentales et protocoles de test (température, vitesse, configuration véhicule, équipements utilisés).
- Les critères de conformité et seuils d'acceptation permettant de valider ou non un système ou un véhicule.
- Les méthodes de documentation, de traçabilité et de validation des résultats d'essais.
- Les exigences de certification et de reconnaissance internationale des résultats d'essais.

Parmi les règlements les plus utilisés dans l'industrie automobile, on peut citer notamment les règlements UNECE tels que ceux relatifs au freinage, à l'éclairage, aux émissions ou encore à la sécurité des occupants. À ces règlements s'ajoutent des cadres régionaux comme les directives européennes, ainsi que des réglementations nationales adaptées à des marchés spécifiques.

L'application rigoureuse de ces textes réglementaires est indispensable pour assurer :

- La sécurité des usagers de la route.
- La réduction de l'impact environnemental des véhicules.
- L'harmonisation internationale des exigences techniques.
- Et la libre circulation des véhicules sur les marchés mondiaux.

Dans ce contexte, la complexité croissante des réglementations, leur évolution continue et leur volume important rendent leur gestion de plus en plus difficile sans outils numériques adaptés. Cela justifie pleinement le besoin de solutions digitales capables de centraliser, structurer et exploiter efficacement ces règlements dans le cadre des processus d'homologation.

2.2. Problématique

La gestion et la consultation des règlements d'homologation représentent un enjeu majeur pour les industriels du secteur automobile. Le contexte actuel se caractérise par plusieurs difficultés :

- **Complexité croissante des documents :** les règlements sont longs, complexes et parfois difficiles à exploiter (plus de 100 pages).
- **Volume important et fragmentation :** les documents sont dispersés entre plusieurs sources (UNECE, directives européennes, réglementations nationales).
- **Difficulté de recherche d'information :** il est difficile d'identifier rapidement une information précise dans des documents volumineux.
- **Évolution continue :** les normes sont régulièrement mises à jour, ce qui complique la veille réglementaire.
- **Manque de structuration et de traçabilité :** les informations ne sont pas centralisées, ce qui complique leur exploitation.
- **Processus inefficaces :** la consultation manuelle est chronophage et sujette aux erreurs.

Dans ce contexte, la digitalisation et la centralisation de la gestion des règlements d'homologation apparaissent comme des leviers essentiels pour :

- Améliorer l'accès à l'information réglementaire
- Réduire les temps de recherche et d'analyse
- Améliorer la fiabilité et la conformité
- Faciliter la collaboration entre les acteurs

3. Solution proposée

Le projet « Segula Homologation 360 » vise à développer une plateforme web intelligente permettant de centraliser, consulter et explorer les règlements d'homologation automobile à travers les fonctionnalités suivantes :

Fonctionnalités principales

- **Centralisation des règlements :** accès unique aux règlements UNECE et autres réglementations applicables
- **Extraction intelligente des données :** analyse automatique des documents PDF pour structurer les informations

- **Consultation structurée :** présentation claire des règlements incluant :
  - Objectifs et portée du règlement
  - Étapes et conditions d'essai
  - Paramètres mesurés et critères de conformité
- **Visualisation 3D interactive :** exploration des composants du véhicule et identification des règlements associés
- **Assistant intelligent :** L'assistant IA basé sur une architecture RAG
- **Gestion documentaire** : bibliothèque centralisée des documents officiels UNECE
- **Gestion des utilisateurs et des rôles :** contrôle d'accès selon les profils
- **Amélioration de la collaboration :** partage facilité de l'information réglementaire

La solution repose sur une architecture moderne en trois couches :

- **Frontend** : application développée en React (TypeScript), offrant une interface intuitive et ergonomique.
- **Backend** : API basée sur Java/Spring Boot, assurant la gestion de la logique métier.
- **Base de données** : système relationnel garantissant la persistance, l'intégrité et la sécurité des données.

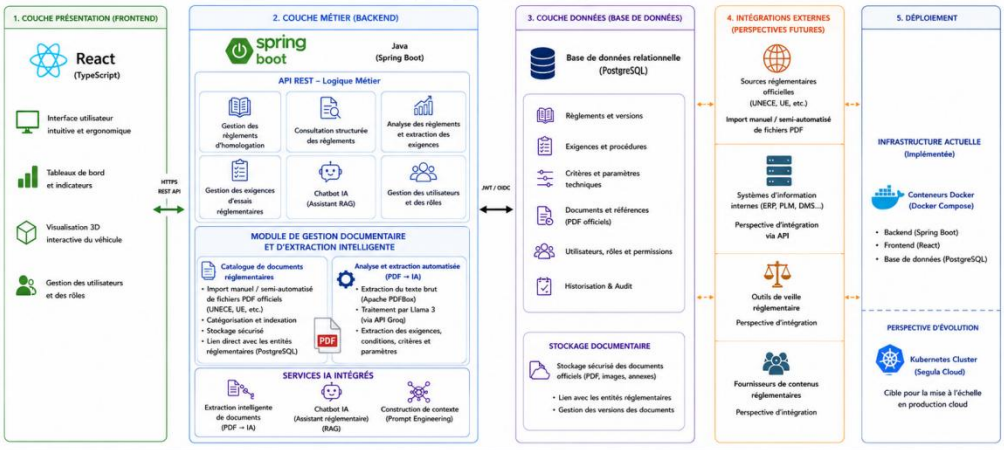


Figure 8 : Architecture de la solution

Elle s'appuie également sur une approche DevOps intégrant :

- La conteneurisation de l'application à l'aide de Docker et Docker Compose.
- L'automatisation des processus de build et de déploiement via GitHub Actions .
- Une architecture évolutive permettant l'intégration future de solutions
- d'orchestration (Kubernetes) et de supervision (Prometheus, Grafana) dans des versions ultérieures.

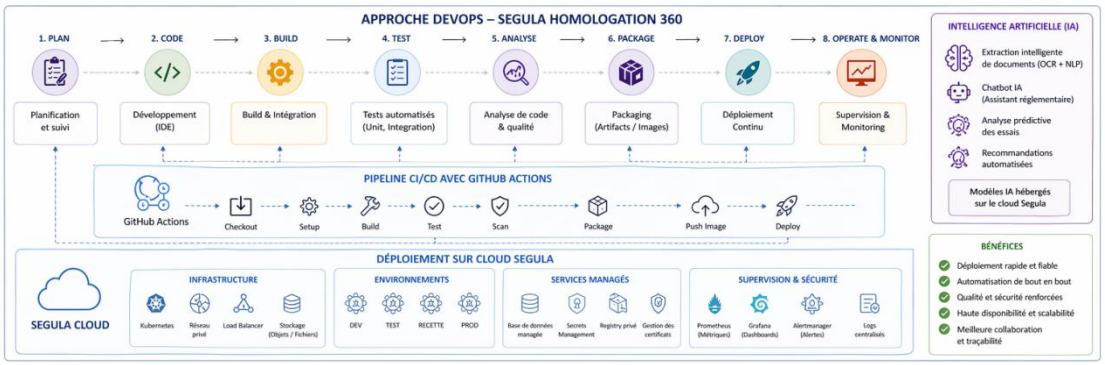


Figure 9 : Approche Devops

4. Analyse détaillée des besoins

4.1. Besoins fonctionnels

À partir de l'analyse des besoins et des objectifs définis pour la plateforme « Segula Homologation 360 », les exigences fonctionnelles suivantes ont été retenues :

- La plateforme doit permettre la gestion centralisée des règlements et documents réglementaires utilisés dans le processus d'homologation.
- La plateforme doit intégrer un mécanisme d'importation et d'analyse automatique des documents PDF afin de faciliter l'extraction et l'exploitation des informations réglementaires.
- La plateforme doit intégrer un assistant intelligent reposant sur une architecture RAG permettant d'interroger les données réglementaires extraites et de générer des réponses pertinentes en fonction du contexte de la requête.
- La plateforme doit permettre la visualisation interactive de modèles tridimensionnels afin d'améliorer l'exploration des composants concernés par les exigences réglementaires.
- La plateforme doit assurer la gestion des utilisateurs, des rôles et des droits d'accès conformément aux exigences de sécurité du système.
- La plateforme doit fournir une interface de pilotage permettant la consultation des informations essentielles relatives aux données réglementaires et aux activités de la plateforme.

4.2. Priorisation des fonctionnalités

Les fonctionnalités développées ont été classées selon leur contribution aux objectifs de la plateforme et aux besoins identifiés lors de l'analyse du contexte métier.

Les fonctionnalités principales comprennent la gestion des règlements, l'importation et l'extraction des données réglementaires depuis les documents PDF, l'assistant intelligent basé sur une architecture RAG ainsi que la gestion des utilisateurs et des rôles. Ces fonctionnalités constituent le cœur de la solution développée.

Les fonctionnalités complémentaires, telles que la visualisation 3D du véhicule et le tableau de bord synthétique, ont pour objectif d'améliorer l'accessibilité des informations réglementaires et l'expérience utilisateur.

Enfin, certaines évolutions futures pourront concerner le renforcement des mécanismes de supervision, l'amélioration des performances de recherche documentaire ou encore le déploiement de la plateforme dans des environnements cloud distribués.

4.3. Besoins non fonctionnels

Outre les aspects fonctionnels, la solution doit respecter les exigences suivantes :

- **Sécurité** : protection des données sensibles et gestion rigoureuse des accès (conformité RGPD).
- **Scalabilité** : capacité à supporter une augmentation du volume de données et d'utilisateurs.
- **Performance** : temps de réponse optimisés et traitement efficace des opérations.
- **Fiabilité** : haute disponibilité, tolérance aux pannes et mécanismes de sauvegarde.
- **Interopérabilité** : intégration avec les systèmes existants (ERP, GED, outils métiers).
- **Ergonomie** : interface intuitive adaptée aux différents profils utilisateurs.
- **Traçabilité** : conservation des historiques et des journaux d'activité.
- **Maintenabilité** : facilité de déploiement et d'évolution grâce aux pratiques DevOps.

5. Contraintes et enjeux

Le projet doit répondre à plusieurs contraintes majeures :

- Respect strict des normes d'homologation automobile.
- Garantie de la confidentialité et de la sécurité des données.
- Adaptation continue aux évolutions réglementaires.
- Facilité d'utilisation pour des profils variés.
- Compatibilité avec les systèmes existants.
- Assurance de la pérennité et de l'évolutivité de la solution.

6. Planification du projet

La planification du projet « Segula Homologation 360 » constitue une étape essentielle permettant d'organiser les différentes phases de réalisation, de structurer les tâches et de garantir le respect des délais.

6.1. Méthodologie adoptée

Dans le cadre de ce projet, une approche structurée basée sur un découpage en phases successives a été adoptée. Cette organisation permet de suivre l'évolution du projet de manière claire, depuis l'analyse des besoins jusqu'au déploiement final.

6.2. Découpage du projet

Le projet a été organisé en plusieurs étapes principales :

- **Analyse des besoins** : identification des exigences fonctionnelles et non fonctionnelles.
- **Conception** : définition de l'architecture et des solutions techniques.
- **Développement** : implémentation des différentes fonctionnalités de la plateforme.
- **Tests et validation** : vérification du bon fonctionnement et correction des anomalies.
- **Déploiement** : mise en production de la solution.

6.3. Outils de planification

Afin d'assurer un suivi efficace du projet, les outils suivants ont été utilisés :

- **Diagramme de Gantt** : utilisé pour planifier les différentes tâches dans le temps, définir leur durée et organiser leur enchaînement ;
- **Git** : utilisé comme système de gestion de versions pour suivre les modifications du code source, assurer la collaboration et garantir la traçabilité des évolutions.

Ces outils ont permis de structurer le travail, de suivre l'avancement du projet et de maintenir une organisation claire des différentes étapes.

6.4. Diagramme de Gantt

Le diagramme de Gantt constitue un outil central dans la planification du projet. Il permet de visualiser l'ensemble des tâches, leur durée ainsi que leur ordre d'exécution.

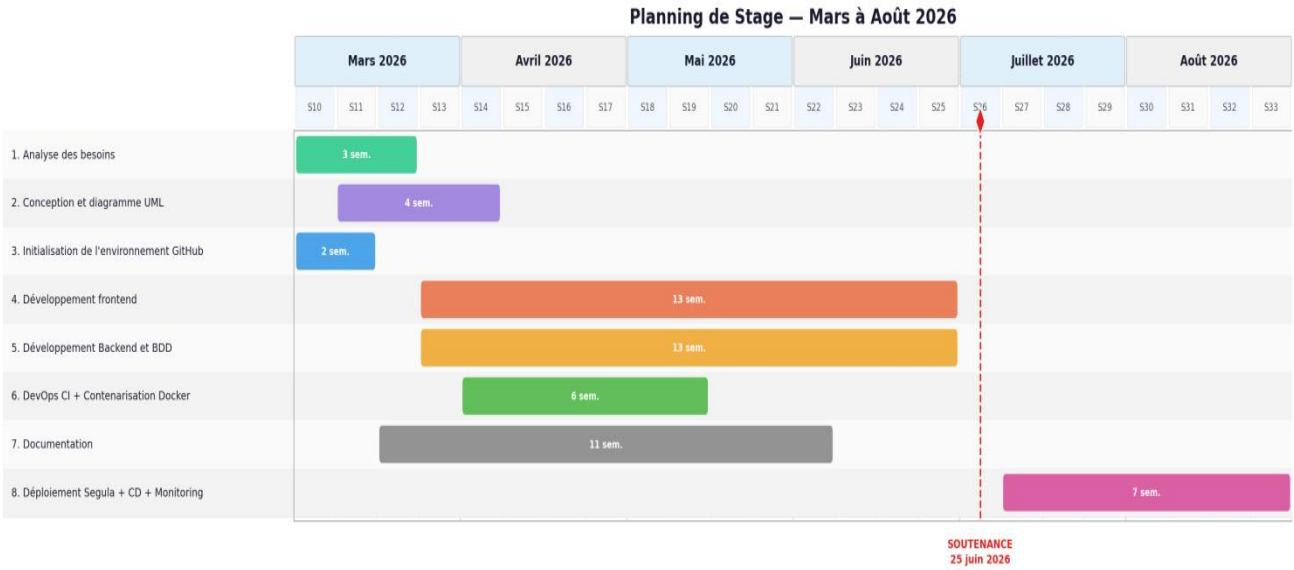


Figure 10 : Diagramme de Gantt

Les principales activités planifiées sont :

- Analyse et spécification des besoins.
- Conception technique et fonctionnelle.
- Développement des modules.
- Phase de tests et validation.
- Déploiement final.

Ce diagramme facilite le respect des délais et offre une vision globale de l'avancement du projet.

6.5. Gestion des risques

La planification du projet a également pris en compte certains risques potentiels, tels que :

- La complexité technique de certaines fonctionnalités.
- Les contraintes liées aux performances et à la sécurité.
- Les éventuels retards dans le développement.

Pour y faire face, des mesures ont été adoptées, notamment :

- Une organisation rigoureuse des tâches.
- Des phases de test régulières.
- Un suivi continu de l'avancement du projet.

7. Conclusion

Ce chapitre a permis d'analyser en profondeur le contexte du projet, d'identifier les principales problématiques liées à la gestion des essais d'homologation et de définir les besoins fonctionnels et non fonctionnels de la solution proposée.

Cette étude constitue le socle de conception de la plateforme « Segula Homologation 360 », dont l'architecture et la réalisation seront détaillées dans le chapitre suivant.

Chapitre 3

Conception de la solution

1. Introduction

Ce chapitre décrit l'approche de conception de la plateforme « Segula Homologation 360 ». Il présente l'architecture logicielle, les choix technologiques, la modélisation des données et des processus, ainsi que les principaux diagrammes UML.

L'objectif est de démontrer la cohérence, la robustesse et l'adéquation de la solution par rapport aux besoins identifiés.

2. Architecture générale du système

La solution s'appuie sur une architecture logicielle en trois couches (3-tiers), organisée comme suit :

- **Couche présentation (Frontend)**

La couche de présentation est constituée d'une application web développée avec React et TypeScript. Elle fournit une interface moderne, ergonomique et responsive permettant aux utilisateurs d'accéder aux différentes fonctionnalités de la plateforme, notamment la gestion des règlements, la gestion documentaire, l'assistant intelligent et la visualisation 3D.

- **Couche métier (Backend)**

La couche métier repose sur une API REST développée en Java avec Spring Boot. Elle centralise l'ensemble de la logique applicative, notamment la gestion des utilisateurs, des règlements, des documents réglementaires, des traitements d'intelligence artificielle ainsi que les mécanismes de sécurité.

Les fonctionnalités d'extraction documentaire et d'assistance intelligente sont implémentées sous forme de services internes intégrés au backend, garantissant une meilleure cohérence de l'architecture et une maintenance simplifiée.

- **Couche données**

La couche de données repose sur PostgreSQL, un système de gestion de base de données relationnelle assurant la persistance, la sécurité et la traçabilité des informations de la plateforme.

Les données stockées concernent notamment les utilisateurs, les rôles, les règlements, les documents réglementaires, les informations extraites des documents ainsi que les retours utilisateurs.

- **Déploiement et conteneurisation**

L'ensemble des composants de la plateforme est conteneurisé à l'aide de Docker afin de garantir la reproductibilité de l'environnement d'exécution et de simplifier le déploiement de l'application.

L'architecture a été conçue de manière modulaire afin de faciliter son évolution future. Des solutions d'orchestration et de supervision telles que Kubernetes, Prometheus et Grafana pourront être intégrées dans des versions ultérieures pour répondre à des besoins de montée en charge et de monitoring avancé.

3. Choix techniques

3.1. Frontend

- **Framework : React**

React a été retenu pour sa capacité à construire des interfaces complexes basées sur des composants réutilisables. Ce choix facilite l'intégration des différents modules de la plateforme, tels que la visualisation 3D, l'assistant intelligent et les tableaux de bord interactifs.

- **Langage : TypeScript**

TypeScript apporte un typage statique permettant d'améliorer la robustesse, la maintenabilité et la qualité globale du code.

- **Outils**

  - Vite pour le développement et la génération des builds.
  - ESLint pour le contrôle qualité du code.
  - Architecture modulaire basée sur des composants réutilisables.

- **Modules principaux**

  - Dashboard
  - Gestion des règlements
  - Gestion documentaire
  - Assistant IA
  - Visualisation 3D basée sur Three.js
  - Administration des utilisateurs

3.2. Backend

- **Framework : Spring Boot (Java)**

Spring Boot a été choisi pour sa robustesse et sa capacité à structurer efficacement les applications d'entreprise. Il facilite la création d'API REST sécurisées et l'intégration avec les différents composants du système.

- **Fonctionnalités principales**

  - API REST
  - Authentification et autorisation
  - Gestion des utilisateurs
  - Gestion des règlements

- Gestion documentaire-
- Services d'intelligence artificielle
- Traitement et extraction documentaire

- **Services internes**

  - **AssistantService :** implémentation d'un assistant intelligent basé sur une architecture RAG (Retrieval-Augmented Generation) permettant l'enrichissement des requêtes avant leur transmission à un modèle de langage externe.
  - **PdfExtractionService :** analyse et extraction automatisée des informations contenues dans les documents PDF réglementaires.
  - **Logique métier :** orchestration des traitements applicatifs et validation des données.

- **Sécurité**

  - Authentification par JWT.
  - Gestion des rôles et des autorisations.
  - Protection des API REST.

- **Collecte réglementaire**

Un module de scraping développé en Python permet de collecter automatiquement les règlements UNECE et leurs métadonnées afin d'alimenter la base documentaire de la plateforme.

3.3. Base de données

- **PostgreSQL**

PostgreSQL a été retenu pour sa fiabilité, son intégrité transactionnelle et ses performances sur les requêtes complexes liées à la gestion réglementaire.

Les données stockées comprennent :

- Utilisateurs et rôles
- Règlements
- Documents réglementaires
- Données extraites
- Feedbacks utilisateurs

- **Administration des données**

Scripts SQL d'initialisation ;

Scripts de migration ;

Sauvegarde et restauration des données.

3.4. DevOps

- **Conteneurisation**

Docker ;

Docker Compose pour l'exécution locale des différents composants.

- **Intégration continue**

GitHub Actions pour l'automatisation du build et des tests.

- **Perspectives d'évolution**

Les technologies suivantes ont été étudiées mais ne sont pas implémentées dans la version actuelle de la plateforme :

- Kubernetes pour l'orchestration des conteneurs.
- Prometheus pour la collecte de métriques.
- Grafana pour la supervision et la visualisation des indicateurs.
- Déploiement multi-instances et montée en charge automatisée.

4. Modélisation UML
4.1. UML

UML (Unified Modeling Language), est un langage de modélisation visuel standardisé utilisé pour représenter la conception de systèmes complexes, principalement dans le développement de logiciels orientés objet. Il se compose d'un ensemble intégré de diagrammes qui permettent de spécifier, visualiser, construire et documenter les artefacts des systèmes logiciels et des processus métier. UML facilite la communication entre les membres d'une équipe de projet en fournissant une notation graphique claire et cohérente pour décrire les structures et le comportement des systèmes.



Figure 11 : Logo UML

4.2. Justification du choix de l'UML

Le choix d'UML comme langage de modélisation pour ce projet repose sur plusieurs avantages significatifs :

- UML offre une représentation visuelle claire qui facilite la compréhension du système à concevoir. Les différents diagrammes permettent d'aborder la conception sous des angles complémentaires, ce qui améliore la qualité globale de l'architecture.

- Les diagrammes UML établissent un pont entre les exigences fonctionnelles et l'implémentation technique. Cette traçabilité est essentielle pour garantir que la solution développée répond effectivement aux besoins exprimés.

- L'approche structurée d'UML aide à détecter les incohérences et les problèmes de conception dès les premières phases du projet, réduisant ainsi les coûts et les délais liés aux corrections tardives.

- Enfin, les modèles UML constituent une documentation technique pérenne qui facilitera la maintenance et l'évolution du système dans le futur.

4.3. Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (Use Case Diagram) est un diagramme UML qui permet de représenter les **fonctionnalités offertes par un système du point de vue des utilisateurs**. Il décrit les interactions entre les acteurs (utilisateurs ou systèmes externes) et le système, en mettant l'accent sur les **services rendus plutôt que sur l'implémentation technique**.

Ce diagramme répond principalement à la question :
"Qui fait quoi dans le système ?"

Il est composé de :

- **Acteurs :** les entités externes qui interagissent avec le système (utilisateur, administrateur, etc.).
- **Cas d'utilisation :** les fonctionnalités ou services fournis par le système.
- **Relations :** liens entre acteurs et cas d'utilisation (association, inclusion **<<include>>**, extension **<<extend>>**, généralisation).

Ainsi, le diagramme de cas d'utilisation permet de mieux comprendre les besoins fonctionnels du système et sert de base pour la conception détaillée.

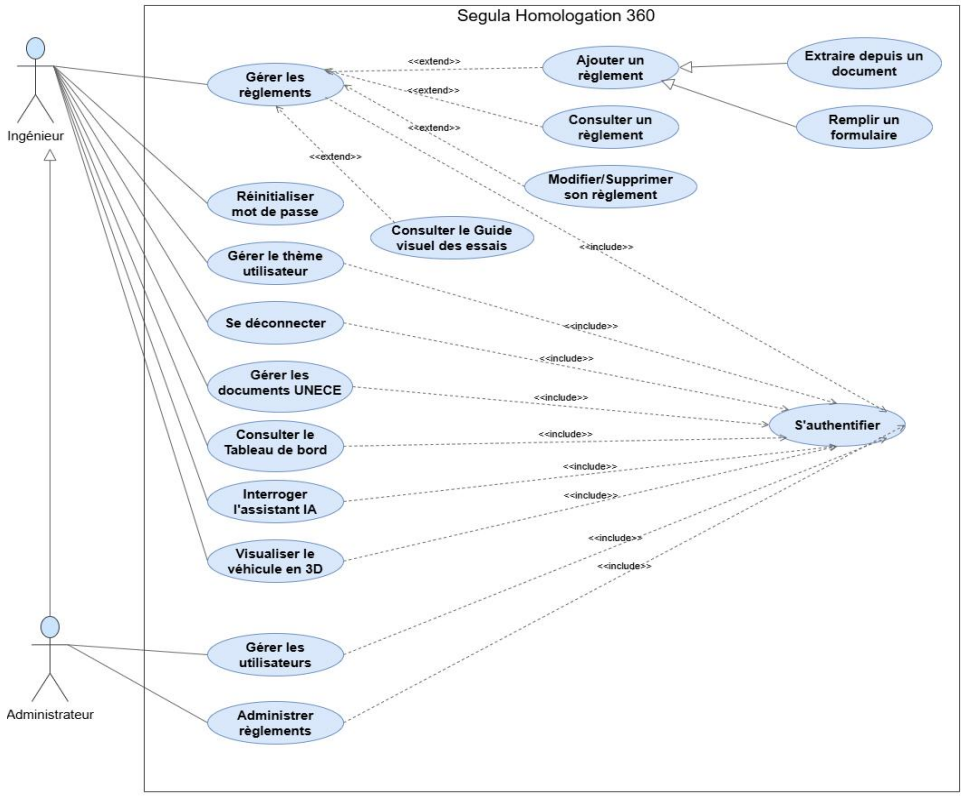


Figure 12 : Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation du système Gestionnaire d'essais d'homologation représente l'ensemble des fonctionnalités offertes aux utilisateurs, principalement l'ingénieur et l'administrateur.

Deux acteurs principaux interviennent :

- **Ingénieur :** utilisateur principal du système, responsable de la gestion et du suivi des règlements ainsi que de l'utilisation des différents services du système.
- **Administrateur :** acteur ayant des privilèges supplémentaires, notamment pour l'administration globale des règlements. Il hérite des fonctionnalités de l'ingénieur grâce à une relation de généralisation.

Fonctionnalités principales :

Le système propose plusieurs fonctionnalités regroupées comme suit :

- **Gestion des accès et authentification :** l'utilisateur peut s'authentifier, se déconnecter et réinitialiser son mot de passe. Ces actions sont obligatoires pour accéder aux autres services.
- **Consultation et assistance :** l'ingénieur peut consulter le guide visuel des essais, les documents UNECE, le tableau de bord, interroger un assistant intelligent basé sur l'IA, ainsi que visualiser les données techniques sous forme de modèle 3D.
- **Gestion des règlements :** cette partie constitue le cœur du système. Elle permet :

    - la consultation des règlements,
    - l'ajout de nouveaux règlements,
    - la modification et suppression de ses propres règlements.
    - Lors de l'ajout, l'utilisateur peut remplir un formulaire ou extraire automatiquement des informations depuis des documents grâce à l'IA.

- **Administration des règlements :** cette fonctionnalité est réservée à l'administrateur, qui peut gérer l'ensemble des règlements du système (modification et suppression globales).
- **Personnalisation :** les utilisateurs peuvent également gérer le thème de l'interface.

Relations entre les cas d'utilisation :

- La relation <<include>> est utilisée pour indiquer que certaines fonctionnalités nécessitent obligatoirement l'authentification (ex : consultation, gestion des règlements).

- La relation <<extend>> est utilisée pour représenter des fonctionnalités optionnelles ou complémentaires, comme l'extraction automatique depuis un document lors de l'ajout d'un règlement.

La relation de généralisation entre l'administrateur et l'ingénieur signifie que l'administrateur hérite de toutes les fonctionnalités de l'ingénieur, avec des droits supplémentaires d'administration.

Ce diagramme permet de visualiser de manière globale les interactions entre les utilisateurs et le système. Il met en évidence les principales fonctionnalités du Gestionnaire d'essais d'homologation ainsi que la répartition des rôles entre ingénieur et administrateur, ce qui facilite la compréhension des besoins fonctionnels avant la phase de conception détaillée.

4.4. Diagramme de classes

Le diagramme de classes ci-dessous modélise l'architecture orientée objet du système de gestion des essais d'homologation. Il s'articule autour de la classe centrale **Reglement**, qui représente une norme d'homologation et constitue l'entité principale du domaine métier.

Cette classe agrège, par composition, plusieurs entités spécialisées : **ConditionEssai**, **ProcedureEssai**, **ParametreMesure** et **CritereConformite**, qui permettent de structurer respectivement les conditions d'essai, les procédures à suivre, les paramètres mesurés ainsi que les critères de conformité. Elle est également associée aux entités **Document**, représentant les pièces justificatives, et **Diapositive**, correspondant aux contenus structurés destinés à la présentation des informations.

Par ailleurs, la classe **Utilisateur** représente les différents acteurs du système. Elle centralise les informations relatives à l'identité, au rôle et aux préférences de l'utilisateur, tout en assurant la gestion des sessions et du statut de connexion. Un utilisateur peut gérer plusieurs règlements, créer plusieurs **Vehicule**, dont les attributs décrivent notamment le modèle, la plateforme, la motorisation et l'URL du modèle 3D, ainsi qu'interagir avec l'assistant intelligent via la classe **MessageChat**. Cette dernière conserve les échanges effectués avec l'assistant, notamment le message utilisateur, la réponse générée par l'intelligence artificielle, le contexte RAG utilisé ainsi qu'un indicateur de pertinence associé à la réponse.

L'ensemble des relations met en évidence une architecture métier cohérente et structurée. Les relations de composition entre **Reglement** et ses sous-entités traduisent un cycle de vie partagé entre les objets, tandis que les associations simples reliant **Utilisateur** aux autres entités représentent des liens fonctionnels indépendants de leur cycle de vie.

Ce diagramme constitue ainsi une référence structurelle importante pour l'implémentation du backend et contribue à garantir une séparation claire des responsabilités entre les différentes composantes du système.

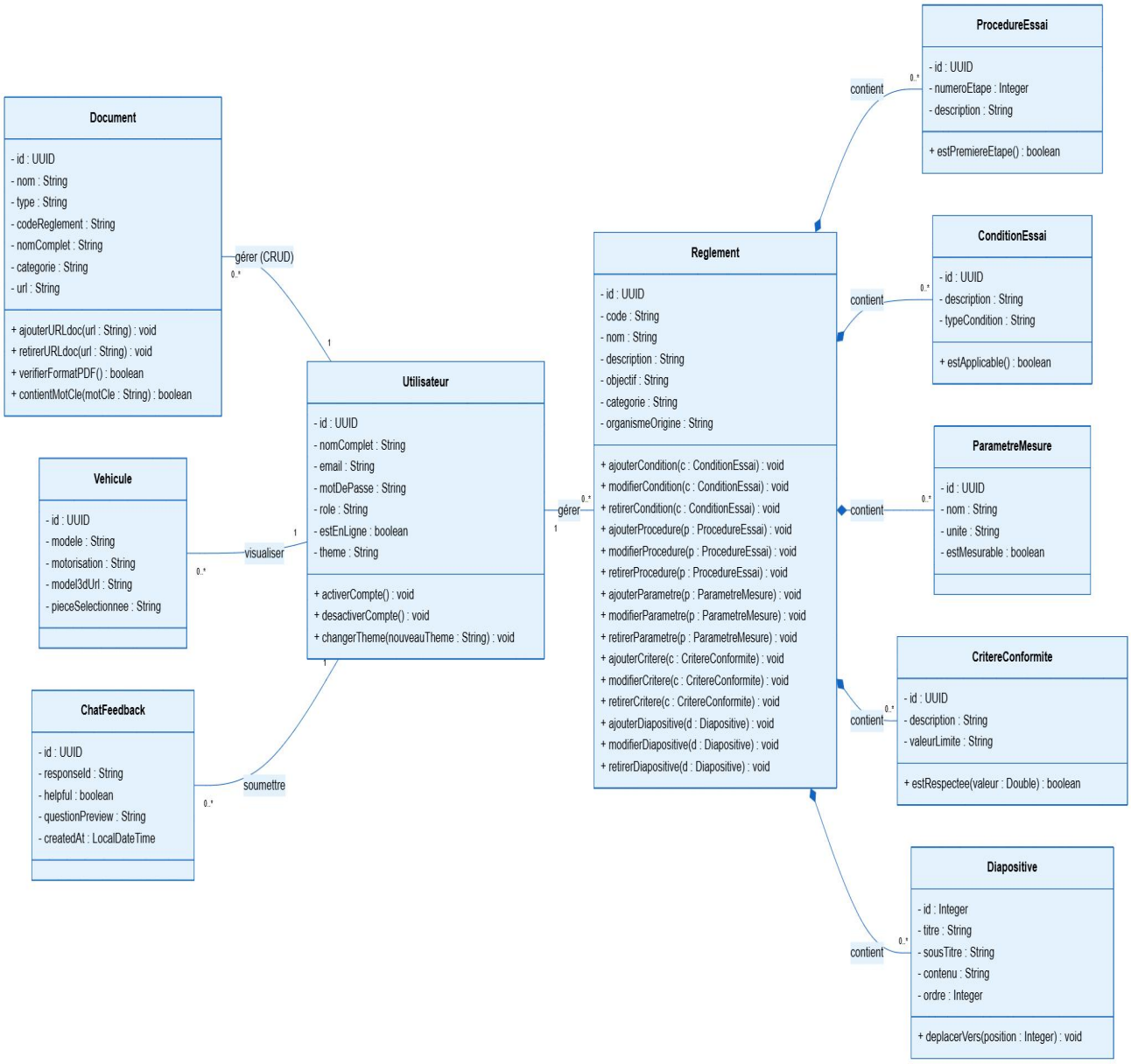


Figure 13 : Diagramme de classes

4.5. Diagramme de séquence

Le diagramme de séquence est un diagramme UML qui représente les interactions entre les différents éléments d'un système selon un ordre chronologique, en montrant comment les composants échangent des messages pour accomplir une fonctionnalité donnée. Il a pour objectif de décrire précisément le déroulement d'un scénario, de modéliser le comportement dynamique du système et de clarifier le rôle de chaque acteur ou composant. Il constitue également un point fort du projet, car il facilite la compréhension globale du fonctionnement de l'application, rend les interactions plus claires et structurées, aide à détecter les erreurs ou incohérences, et améliore la communication entre les membres de l'équipe.

4.5.1. Connexion à l'interface

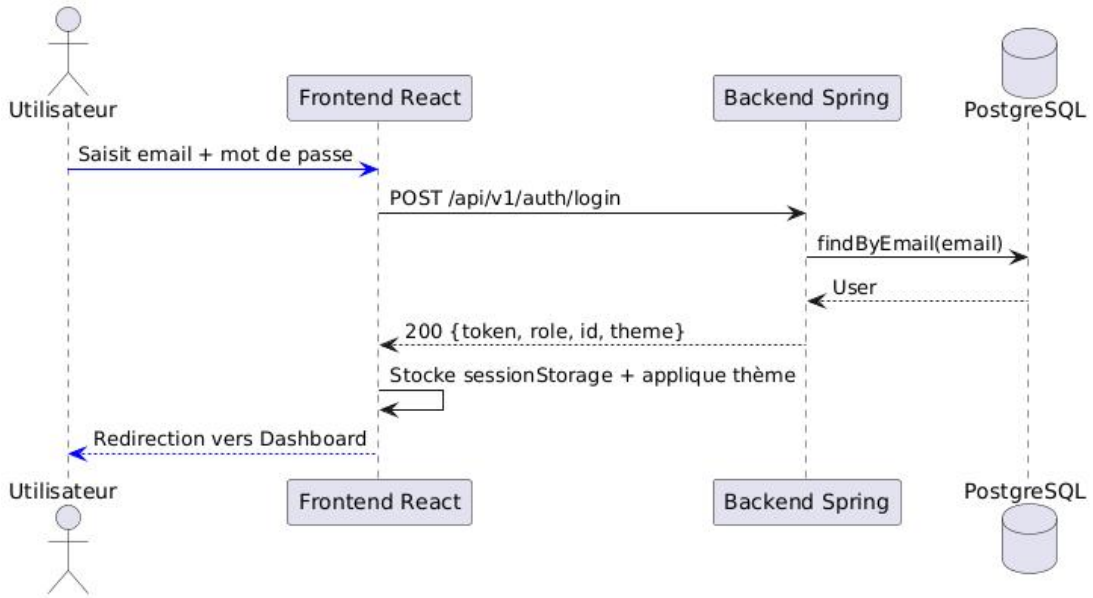


Figure 14 : Diagramme de séquence - Connexion à l'interface

4.5.2. Interface « Mot de passe oublié »

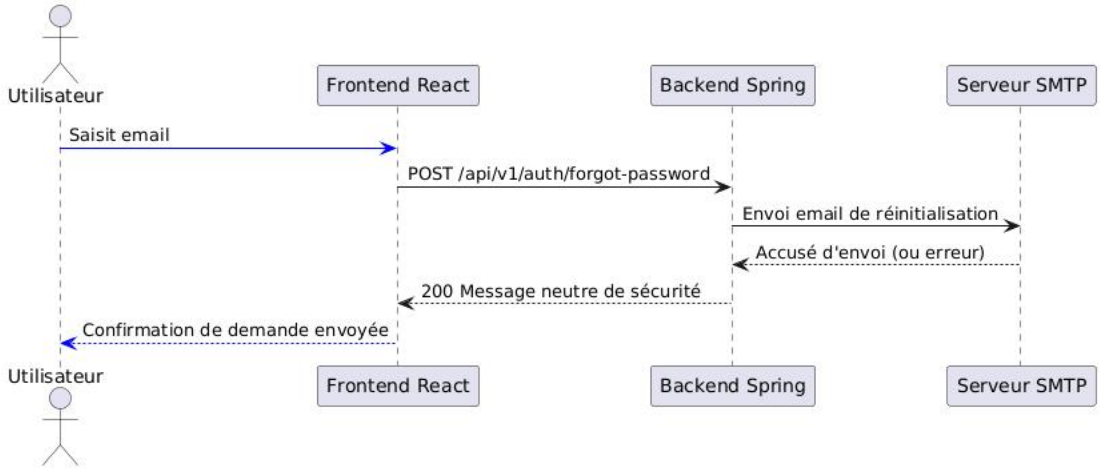


Figure 15 : Diagramme de séquence - Mot de passe oublié

4.5.3. Interface Dashboard

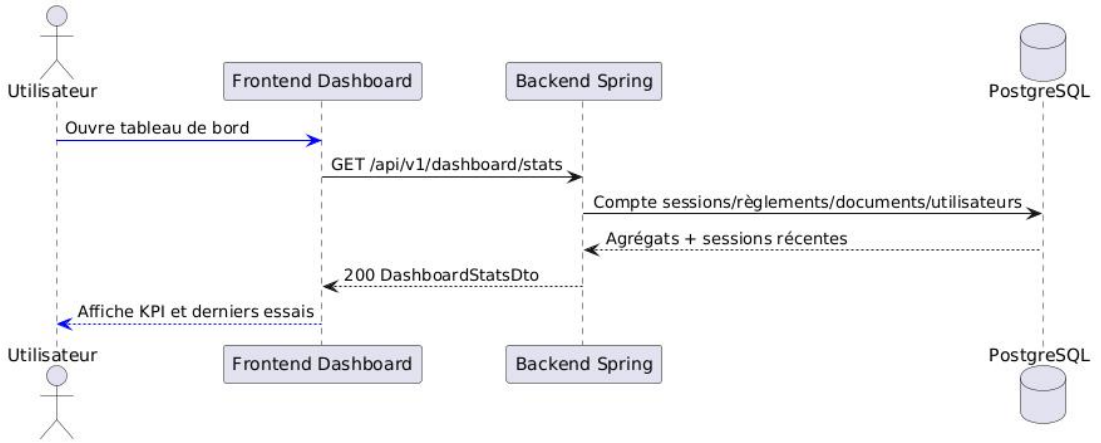


Figure 16 : Diagramme de séquence - Interface Dashboard

4.5.4. Affichage des règlements

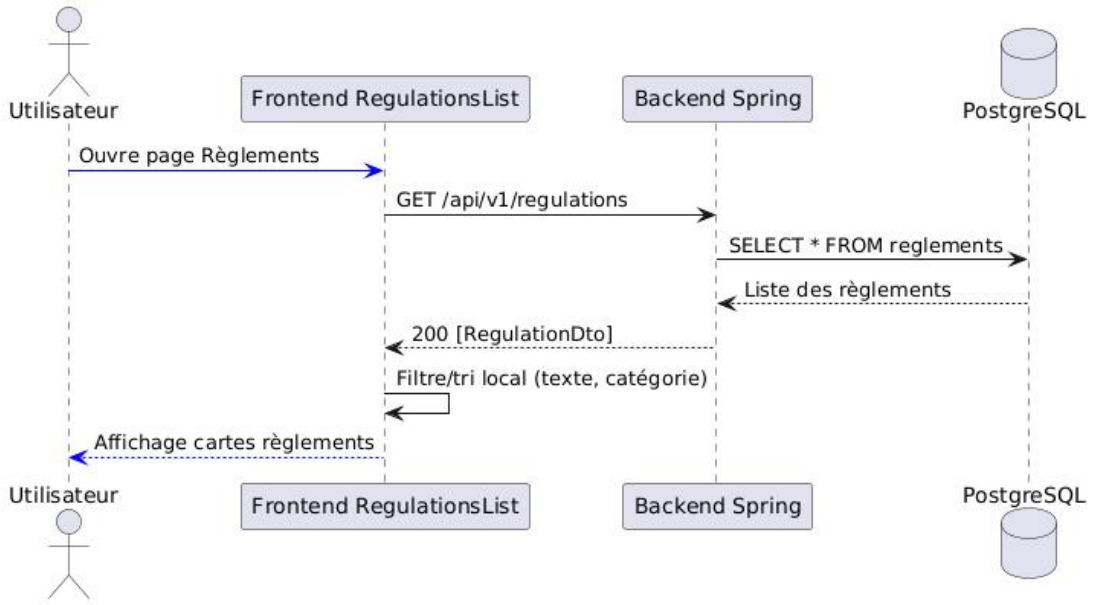


Figure 17 : Diagramme de séquence - Affichage des règlements

4.5.5. Consultation d'un règlement

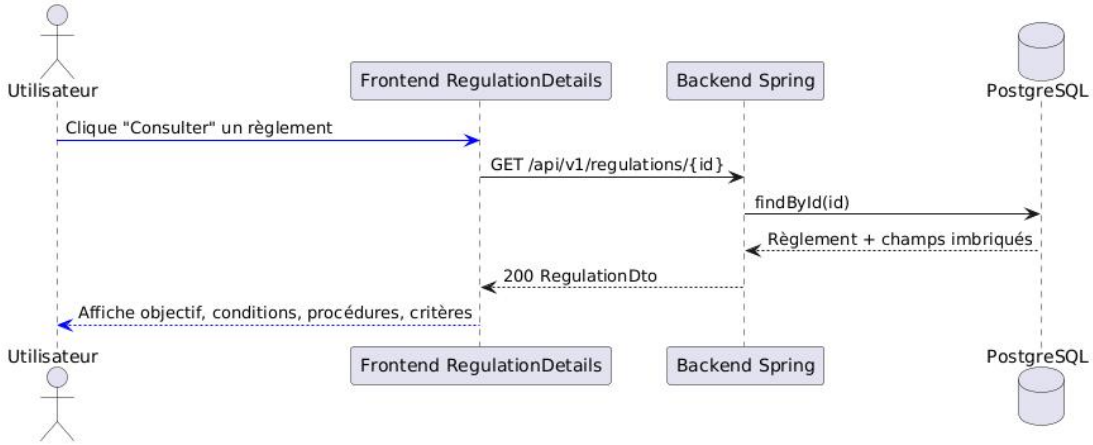


Figure 18 : Diagramme de séquence - Consultation d'un règlement

4.5.6. Création / modification manuelle

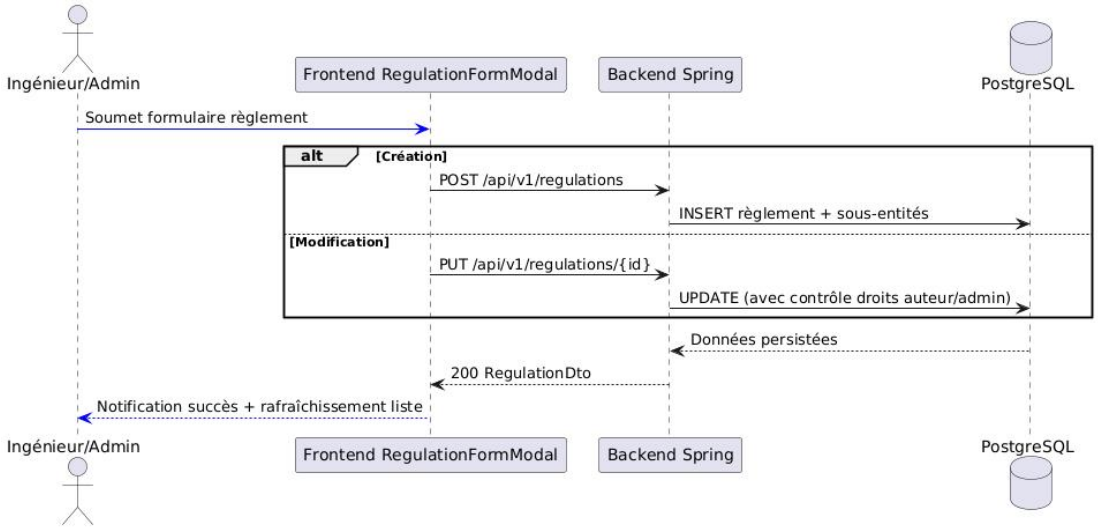


Figure 19 : Diagramme de séquence - Création / Modification manuelle

4.5.7. Extraction PDF par IA

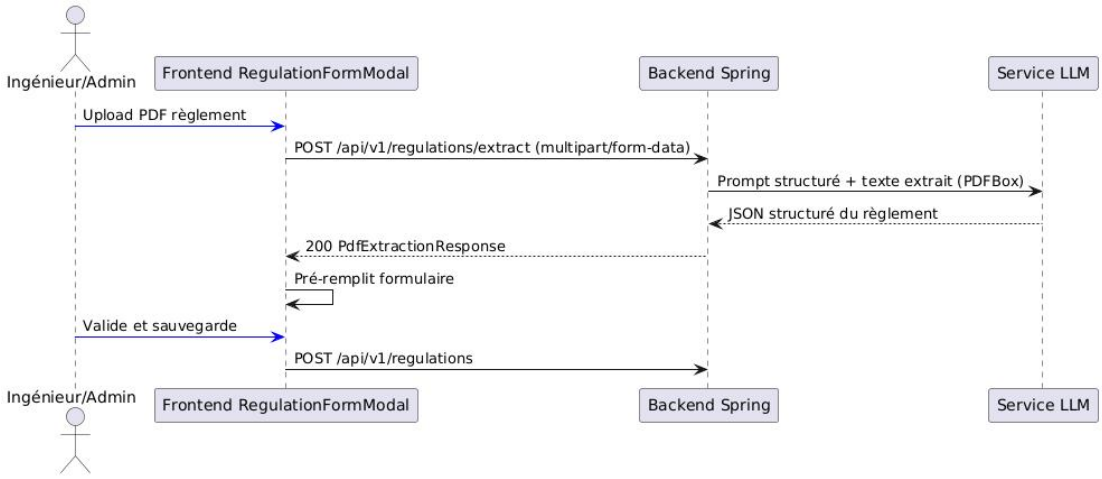


Figure 20 : Diagramme de séquence - Extraction PDF par IA

4.5.8. Visualisation 3D

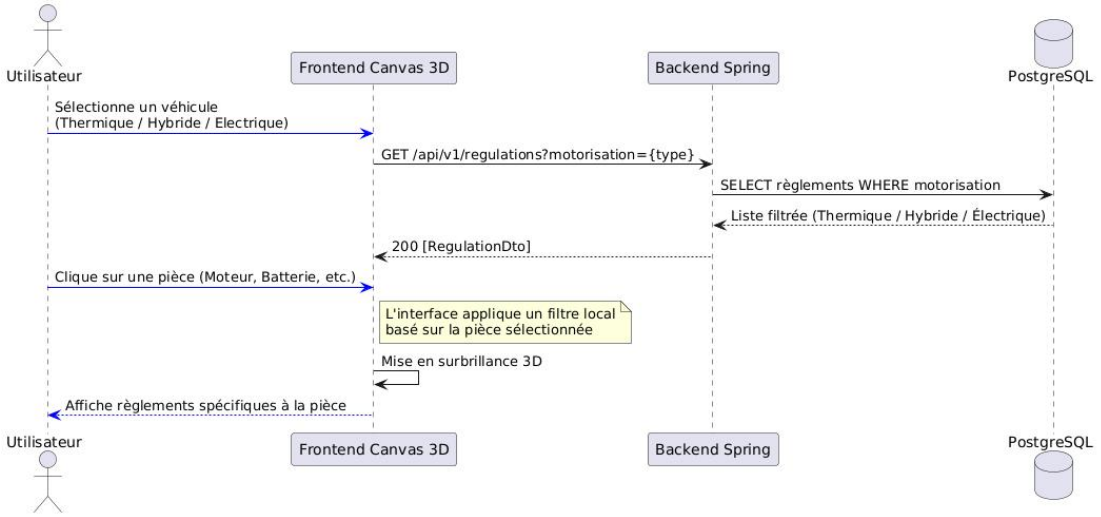


Figure 21 : Diagramme de séquence - Visualisation 3D

4.5.9. Assistant IA

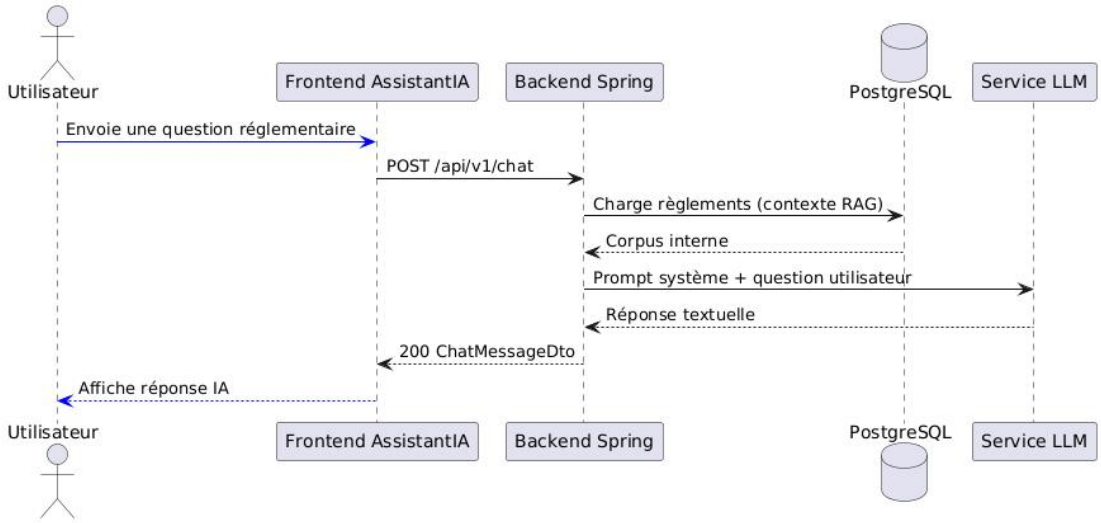


Figure 22 : Diagramme de séquence - Assistant IA

4.5.10. Interface administrateur : Gestion des utilisateurs

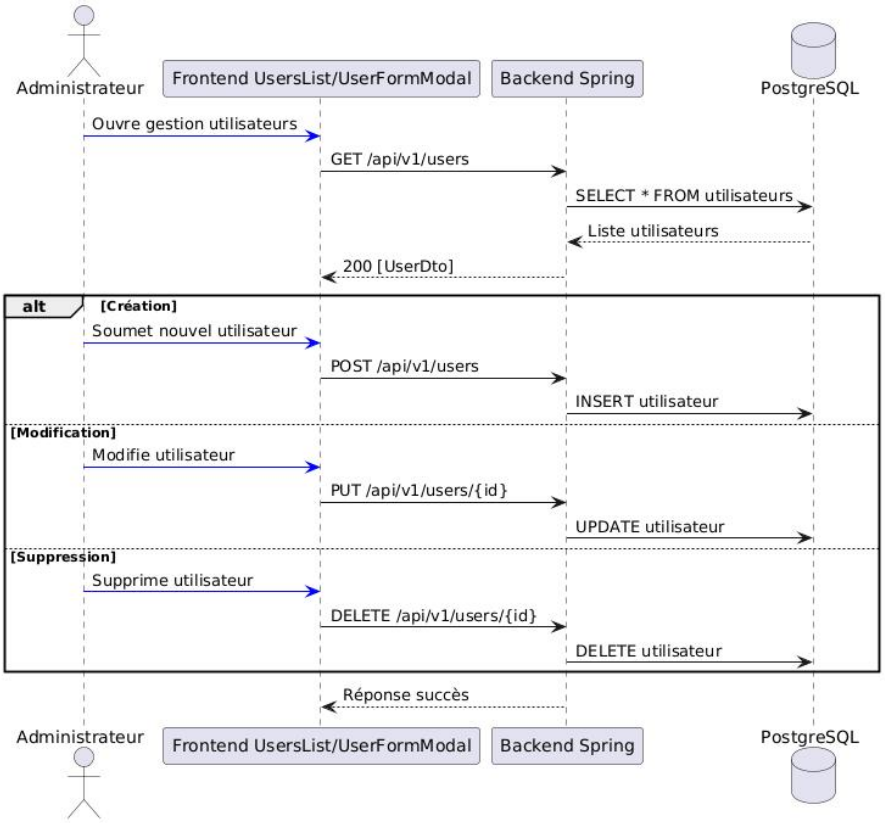


Figure 23 : Diagramme de séquence - Interface administration : Gestion des utilisateurs

4.5.11. Interface des documents réglementaires

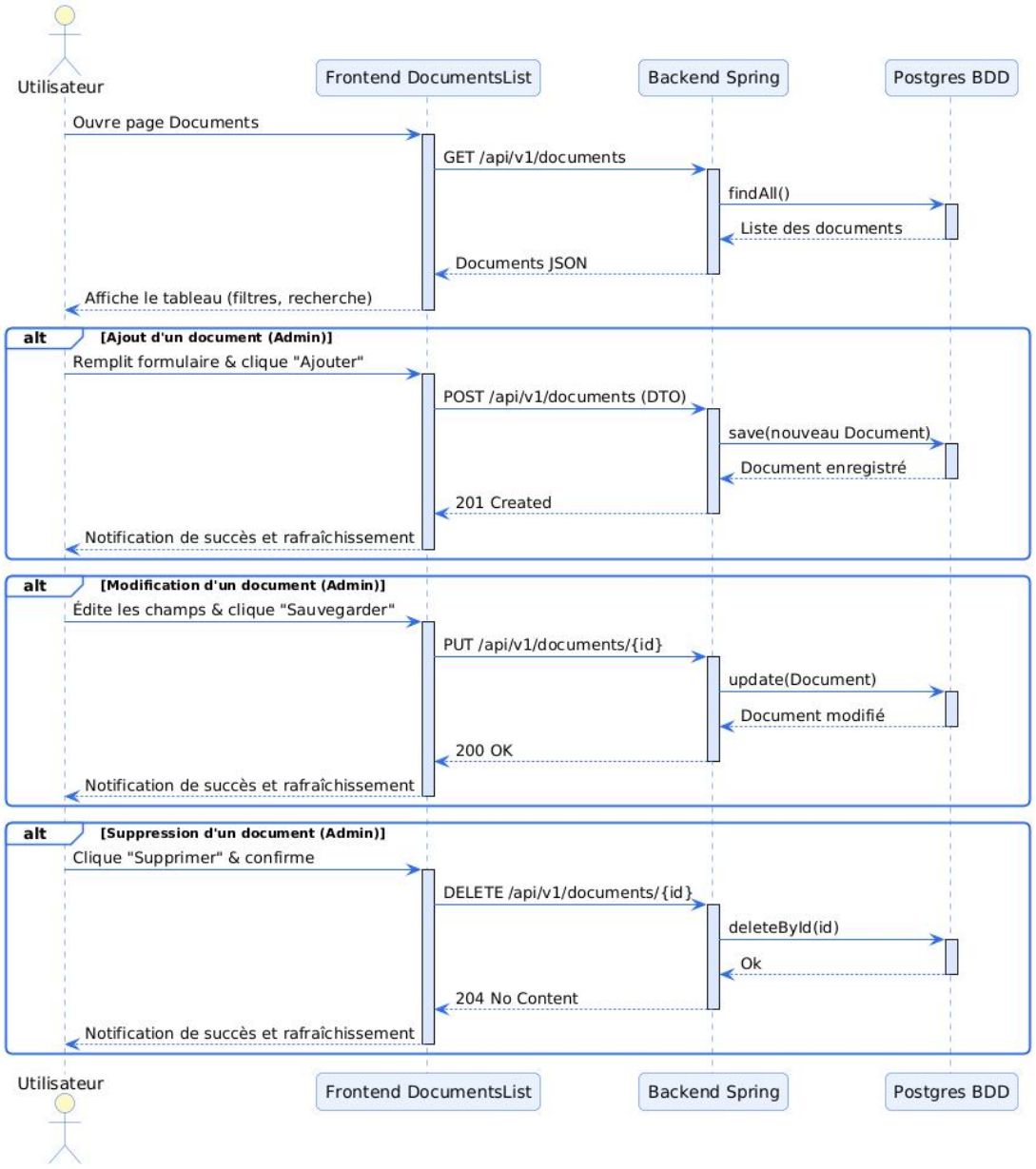


Figure 24 : Diagramme de séquence - Interface des documents réglementaires

4.5.12. Déconnexion et thème

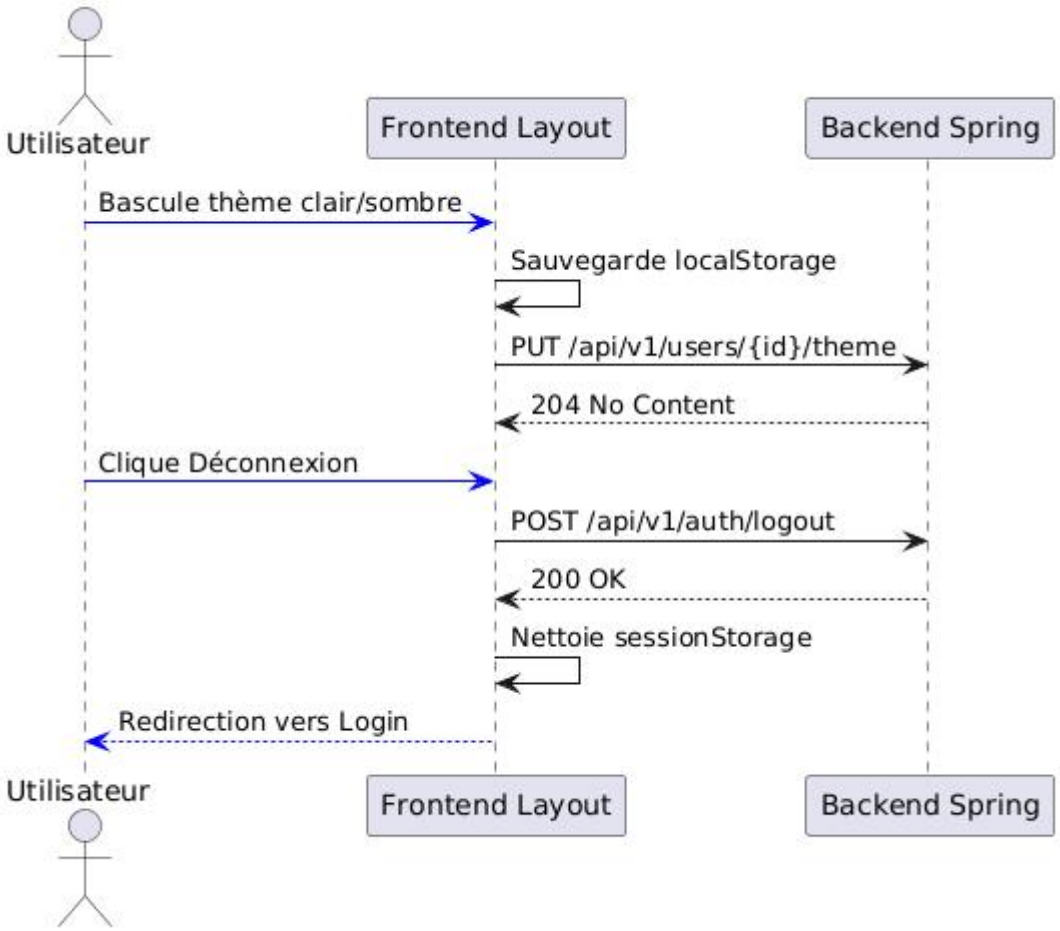


Figure 25 : Diagramme de séquence - Déconnexion et thème

4.6. Diagramme d'activité

Le diagramme d'activité est un diagramme UML qui permet de représenter le déroulement des processus métier ou des flux de travail au sein du système. Il met en évidence les différentes étapes d'un processus, les actions effectuées, ainsi que les décisions et les interactions entre les acteurs et le système.

Contrairement au diagramme de séquence qui se concentre sur les échanges entre objets, le diagramme d'activité offre une vision globale et dynamique du fonctionnement du système, en décrivant le flux des opérations du début à la fin.

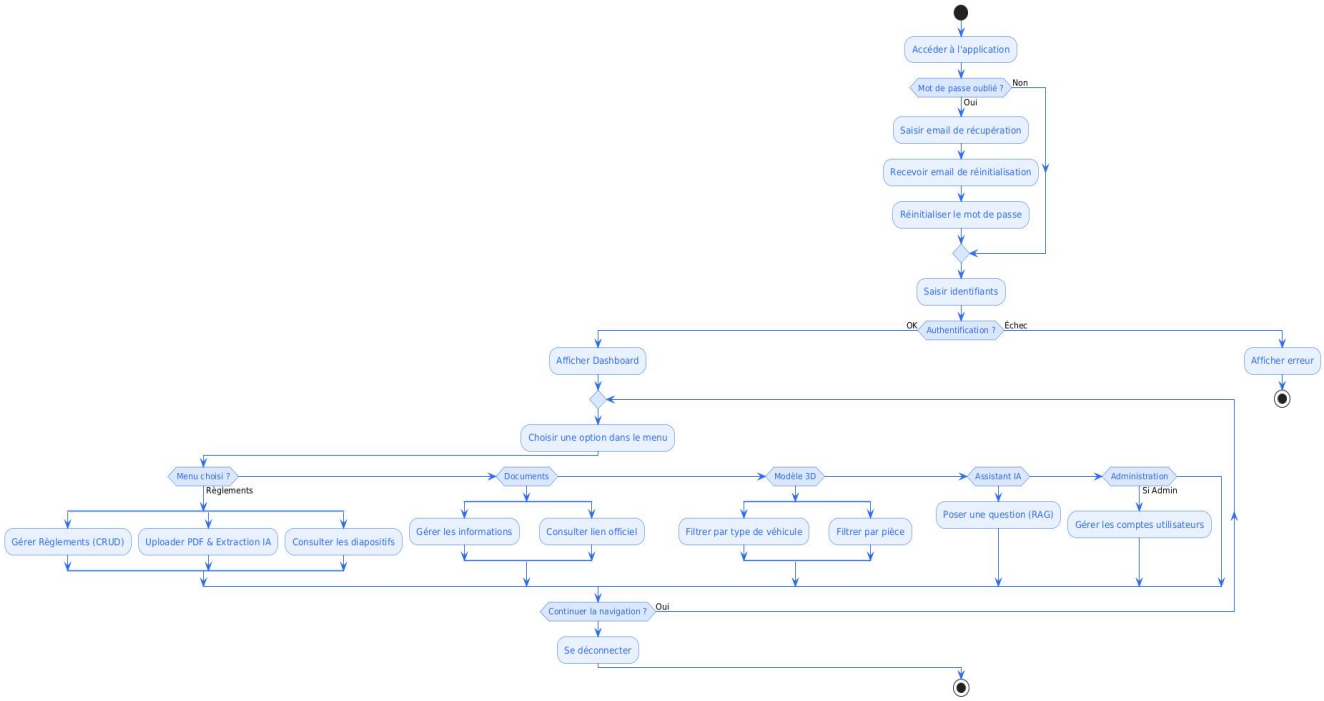


Figure 26 : Diagramme d'activité

Le diagramme d'activité présenté illustre le fonctionnement global de la plateforme Segula Homologation 360, depuis l'authentification de l'utilisateur jusqu'à l'utilisation des différents modules, ainsi que la communication avec le backend.

Phase d'authentification

Le processus débute par l'accès à l'application :

- L'utilisateur saisit son email et son mot de passe.

- En cas d'oubli, il peut déclencher la fonctionnalité **mot de passe oublié**, qui envoie un email via le backend.
- Si les identifiants sont valides, un token JWT est généré et l'utilisateur est authentifié.
- Sinon, une erreur (401) est affichée.

Accès au dashboard

Après authentification :

- La session est stockée côté frontend.
- L'utilisateur accède au dashboard.
- Les indicateurs (KPI) sont récupérés via une API backend.

Navigation entre les modules

L'utilisateur peut choisir différents modules via le menu latéral :

- **Module Réglementation :**
    - Consultation des règlements.
    - Filtrage et affichage des détails.
    - Création de nouveaux règlements (manuel ou via upload PDF).
    - Vérification et validation des données.
- **Module Simulation 3D :**
    - Chargement d'un modèle (ex : pneus).
    - Lancement de simulations.
    - Affichage du verdict (conforme ou non).
- **Assistant IA :**
    - L'utilisateur pose une question.
    - Le backend traite la requête via un module IA (RAG + LLM).
    - Une réponse est générée et affichée.
- **Module Administration :**
    - Gestion des utilisateurs (CRUD).
    - Accès restreint selon le rôle.
    - Refus d'accès si l'utilisateur n'est pas autorisé.

Interaction avec le backend

Tout au long du processus :

- Les actions utilisateur déclenchent des appels API REST (GET, POST).
- Le backend traite les requêtes (authentification, récupération de données, traitement IA, simulation, etc.).
- Les résultats sont renvoyés au frontend pour affichage.

Gestion des actions et notifications

- Les actions importantes (création, modification) génèrent des notifications de succès.
- Les erreurs sont gérées et affichées à l'utilisateur.

Déconnexion

- L'utilisateur peut se déconnecter à tout moment.
- La session est nettoyée.
- Le backend marque l'utilisateur comme hors ligne.

4.7. Diagramme d'architecture technique

L'architecture actuelle de la plateforme « Segula Homologation 360 » repose sur une organisation en trois couches principales : une couche de présentation, une couche métier et une couche de persistance des données.

La couche de présentation est implémentée à l'aide de React et accessible via un navigateur web. Elle prend en charge l'affichage des différentes interfaces de l'application, notamment la gestion des règlements, la gestion documentaire, l'assistant intelligent ainsi que la visualisation 3D. Les interactions avec le backend sont assurées via des API REST.

La visualisation 3D est intégrée directement au sein du frontend. Elle s'appuie sur des modèles tridimensionnels au format GLB stockés localement et rendus à l'aide de bibliothèques JavaScript spécialisées telles que Three.js. Cette fonctionnalité ne repose donc sur aucun service externe ou microservice dédié.

La couche métier est développée avec Spring Boot. Elle centralise la logique applicative, la gestion des utilisateurs, la sécurité (authentification et gestion des rôles), ainsi que l'orchestration des traitements liés aux documents et à l'assistant intelligent.

Les fonctionnalités d'extraction documentaire sont implémentées via des services internes permettant l'analyse et le traitement des fichiers PDF.

Les données extraites sont ensuite structurées et intégrées dans la base de données afin de faciliter leur exploitation.

L'assistant intelligent repose sur une architecture de type RAG (Retrieval-Augmented Generation). Les requêtes utilisateur sont enrichies à partir des données réglementaires structurées issues du processus d'extraction, puis transmises à un modèle de langage accessible via une API externe. Afin de garantir la légèreté du système, l'historique complet des échanges n'est pas persisté en base de données ; seules certaines informations, telles que les retours utilisateurs (feedbacks), peuvent être enregistrées.

La couche de persistance repose sur PostgreSQL. Elle assure le stockage des données relatives aux utilisateurs, aux rôles, aux règlements, aux documents importés ainsi qu'aux résultats des traitements d'extraction.

Enfin, l'ensemble des composants de la plateforme est conteneurisé à l'aide de Docker, ce qui permet de garantir la reproductibilité de l'environnement d'exécution et de faciliter le déploiement en environnement de développement.

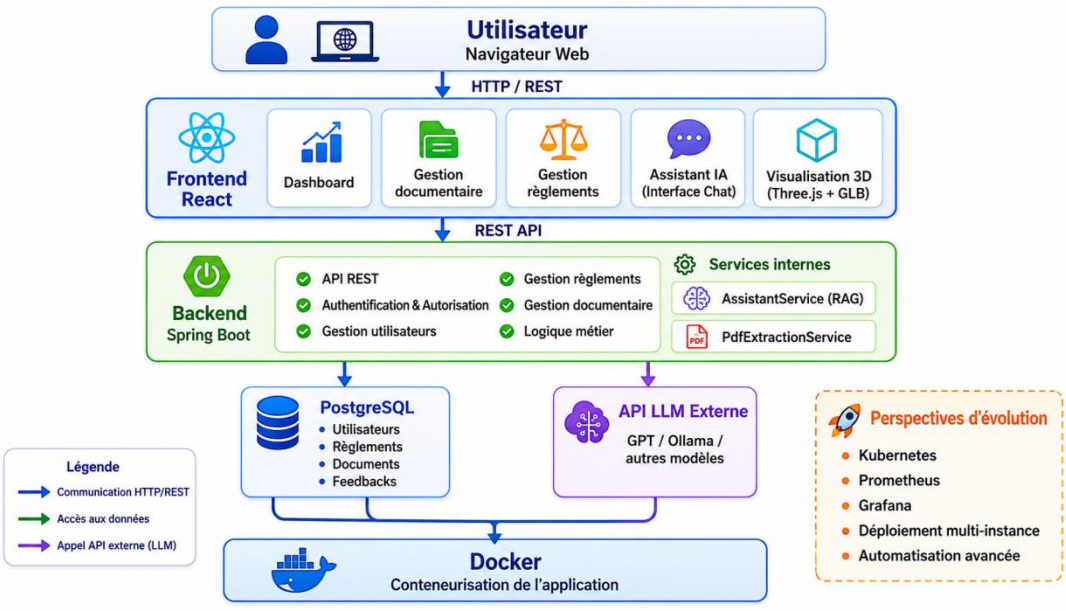


Figure 27 : Diagramme d'architecture technique

5. Gestion documentaire et collecte réglementaire

- **Scraping des règlements UNECE :** un module Python permet de collecter automatiquement les URLs réglementaires et métadonnées afin d'alimenter la base documentaire.
- **Bibliothèque documentaire :** intégrée à la plateforme, elle permet la recherche, la consultation et la gestion des documents réglementaires via des opérations CRUD, avec accès aux sources officielles.
- **Analyse et structuration :** les documents sont classés selon leur thématique, version et domaine d'application, facilitant la veille réglementaire et la conformité.
- **Lien avec les essais :** chaque essai d'homologation est relié aux règlements concernés, garantissant la traçabilité et la justification réglementaire.

6. Sécurité et gestion des accès

- Authentification via login et mot de passe
- Gestion des rôles (administrateur, ingénieur, manager, etc.)
- Sécurisation des API et des données sensibles
- Journalisation et audit des actions utilisateurs

7. Extensibilité et évolutivité

L'architecture de « Segula Homologation 360 » est conçue pour être modulaire et évolutive. La séparation entre frontend, backend et base de données permet de faire évoluer chaque composant indépendamment.

Les modules d'intelligence artificielle, intégrés dans le backend, sont configurables et peuvent utiliser différents services LLM externes, ce qui facilite leur amélioration sans modifier l'architecture globale.

Les fonctionnalités d'extraction et de visualisation 3D sont internes à l'application, garantissant une meilleure maîtrise et une plus grande flexibilité d'évolution.

L'utilisation de Docker facilite le déploiement et la reproductibilité de l'environnement d'exécution de la plateforme. Dans une perspective d'évolution, l'intégration d'une solution d'orchestration telle que Kubernetes permettra d'améliorer la montée en charge et la disponibilité du système.

Ainsi, la plateforme peut facilement intégrer de nouvelles fonctionnalités, s'adapter aux besoins futurs et évoluer sans impact majeur sur le système existant.

8. Conclusion

La conception de la plateforme « Segula Homologation 360 » repose sur une architecture modulaire et des choix technologiques solides, garantissant performance, sécurité et évolutivité. Les choix technologiques retenus ont été effectués selon plusieurs critères : performance, maintenabilité, facilité d'intégration et adéquation avec les contraintes industrielles du domaine de l'homologation automobile. Les modèles UML et les schémas présentés confirment la cohérence de la solution et sa capacité à répondre aux exigences métier. Le chapitre suivant sera consacré à l'implémentation et à la réalisation des différents modules.

Chapitre 4

Réalisation et implémentation

1. Introduction

Ce chapitre présente la réalisation pratique de la plateforme « Segula Homologation 360 ». Il met en évidence l'implémentation des différentes couches de l'application, les technologies utilisées, ainsi que les interfaces utilisateur finalisées. Les captures d'écran des interfaces et les logos des outils et technologies sont intégrés afin d'illustrer concrètement les choix réalisés au cours du développement.

2. Outils et technologies

Dans le cadre de ce projet, un ensemble d'outils, de technologies et de bibliothèques a été utilisé, couvrant le frontend, le backend, la base de données ainsi que le déploiement et le développement.

2.1. Langages de programmation

- **Java 17 (backend)**

Langage backend utilisé pour développer les API REST et la logique métier, compilé et géré via Maven.



Figure 28 : Logo de Java

- **TypeScript (frontend)**

Utilisé pour le frontend afin d'assurer un typage statique des composants React et améliorer la maintenabilité du code.



Figure 29 : Logo de TypeScript

- **JavaScript (**scripts frontend et de build)

Utilisé pour certains scripts frontend, le tooling et des fonctionnalités complémentaires côté client.



Figure 30 : Logo de JavaScript

- **SQL (PostgreSQL)**

Utilisé pour écrire les requêtes, définir les schémas et gérer les migrations dans PostgreSQL via Flyway.



Figure 31 : Logo de PostgreSQL

- **Python (scripts d'extraction et utilitaires)**

Utilisé pour automatiser l'extraction, la transformation et l'import de données réglementaires (scripts dans backend/).



Figure 32 : Logo de Python

2.2. Frontend

- **React (v19)**

Utilisé pour construire une interface dynamique permettant la navigation, l'affichage des règlements et l'interaction utilisateur.

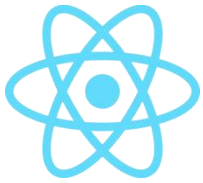


Figure 33 : Logo de React

- **Vite (serveur de développement et build)**

Utilisé comme serveur de développement rapide et outil de build pour optimiser le chargement du frontend.



Figure 34 : Logo de Vite

- **Tailwind CSS (mise en forme)**

Utilisé pour styliser rapidement l'interface utilisateur avec une approche basée sur des classes utilitaires.



Figure 35 : Logo de Tailwind CSS

- **Three.js / react-three-fiber (visualisation 3D)**

Utilisé pour intégrer un modèle 3D du véhicule, permettre l'interaction et la sélection de zones (pneus, moteur, éclairage).

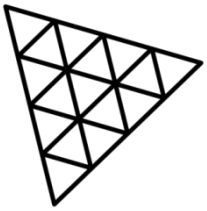


Figure 36 : Logo de ThreeJS

- **Axios (communication HTTP)**

Utilisé pour communiquer avec le backend via des requêtes HTTP (authentification, récupération des règlements, simulation).



Figure 37 : Logo de Axios

- **React Router (gestion du routage)**

Utilisé pour gérer la navigation entre les différentes pages de l'application.



Figure 38 : Logo de React Router

2.3. Backend

- **Spring Boot**

Utilisé comme framework principal pour structurer et lancer l'application backend.



Figure 39 : Logo de SpringBoot

- **Spring Web**

Utilisé pour exposer des API REST permettant la communication avec le frontend.

- **Spring Security**

Utilisé pour sécuriser l'application via l'authentification et la gestion des rôles (Ingénieur, Admin).

- **Spring Data JPA / Hibernate (persistance des données)**

Fournit l'abstraction de persistance et le mapping objet-relationnel pour accéder et manipuler les entités métier en base de données.

- **Flyway (gestion des migrations de base de données)**

Gère le versioning et l'exécution des migrations SQL afin d'assurer la cohérence du schéma entre environnements.



Figure 40 : Logo de Flyway

- **Maven (gestion des dépendances et build)**

Utilisé pour gérer les dépendances et automatiser le build du projet backend.



Figure 41 : Logo de Maven

- **PDFBox (extraction et traitement de documents PDF)**

Utilisé pour extraire et traiter les informations des documents PDF réglementaires.



Figure 42 : Logo de PDFBox

2.4. Conteneurisation et déploiement

- **Docker (création de conteneurs)**

Utilisé pour conteneuriser le backend, le frontend et la base de données afin d'assurer la portabilité.

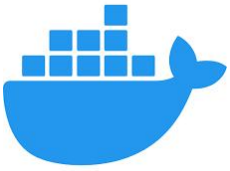


Figure 43 : Logo de Docker

- **Docker Compose (orchestration en local)**

Utilisé pour orchestrer l'ensemble des services en local (backend, frontend, base).

- **Nginx (exposition du frontend et proxy)**

Utilisé pour servir le frontend et agir comme proxy inverse vers les API backend (/api).



Figure 44 : Logo de Nginx

- **Kubernetes (non implémenté dans cette version, prévu pour une évolution future)**

Envisagé pour déployer et scaler l'application dans un environnement cloud.



Figure 45 : Logo de Kubernetes

2.5. DevOps, CI et monitoring

- **Git / GitHub (gestion du code source)**

Utilisés pour la gestion du code source, le versioning et la collaboration.



Figure 46 : Logo de Git

- **GitHub Actions (intégration continue : build et tests)**

Utilisé pour automatiser les pipelines de build, de tests et de déploiement.



Figure 47 : Logo de github

- **Prometheus (non implémenté dans cette version, prévu pour une évolution future)**

Envisagé pour collecter des métriques sur les performances de l'application.



Figure 48 : Logo de Prometheus

- **Grafana (non implémenté dans cette version, prévu pour une évolution future)**

Envisagé pour visualiser les métriques via des dashboards.



Figure 49 : Logo de Grafana

2.6. Intelligence artificielle

- **LLM**

Utilisé pour implémenter un assistant conversationnel basé sur un modèle LLM externe.

- **RAG (Retrieval-Augmented Generation)**

Utilisé pour améliorer la fiabilité en combinant recherche dans les documents réglementaires et génération de réponses.

- **Groq (API LLM)**

Groq est une plateforme cloud d'inférence IA qui fournit un accès aux modèles de langage avancés, notamment Llama 3 et ses variantes. Contrairement aux solutions d'IA traditionnelles, Groq a développé une architecture propriétaire optimisée pour une inférence ultra-rapide (inference engine) appelée "LPU" (Language Processing Unit).

2.7. Ressources 3D

- **Sketchfab**

Utilisé pour rechercher et intégrer des modèles 3D de véhicules au format .gltf / .glb.

2.8. Tests et qualité

- **JUnit**

Utilisé pour tester la logique métier et assurer la fiabilité du backend.

- **Surefire (Maven)**

Utilisé pour exécuter automatiquement les tests lors du build.

- **ESLint**

Utilisé pour garantir la qualité et la cohérence du code frontend.

- **TypeScript Compiler**

Utilisé pour vérifier les types et éviter les erreurs avant exécution.

2.9. Scripts et utilitaires

- **Scripts Python** utilisés pour automatiser l'import et le traitement des données réglementaires.
- **Scripts SQL** utilisés pour initialiser la base de données et injecter des données de test.

3. Outils bureautiques et ressources externes

En complément des outils techniques, plusieurs outils bureautiques et sources externes ont été utilisés pour la documentation, la collecte des normes et la production des livrables.

3.1. Outils bureautiques

- Microsoft Word (rédaction du rapport)
- PowerPoint / Google Slides (présentations)

3.2. Navigation et documentation

- Navigateurs web (Chrome, Microsoft Edge)
- Lecteurs PDF pour consultation des normes

3.3. Sources officielles

- Base de données des règlements UNECE
- Sites gouvernementaux (ex : Legifrance)
- Sketchfab (modèles 3D)

3.4. Tests d'API

- Postman / Insomnia

3.5. Environnement de développement

- Visual Studio Code
- Docker Desktop
- pgAdmin / DBeaver

L'ensemble de ces outils a permis de mettre en place un environnement complet, couvrant le développement, le déploiement, la gestion des données et l'intégration d'un assistant intelligent, tout en facilitant la productivité et la qualité du projet.

4. Présentation des interfaces utilisateur

Cette section présente les principales interfaces réalisées au cours du développement de la plateforme « Segula Homologation 360 ». Chaque capture d'écran illustre une fonctionnalité clé du système.

4.1. Interface de connexion

L'interface de connexion constitue le point d'entrée sécurisé de la plateforme. Elle permet à l'utilisateur de s'authentifier via ses identifiants (email et mot de passe). Le système génère un token JWT en réponse, stocké en session pour les appels API ultérieurs. Une option de récupération de mot de passe est également accessible depuis cette page.

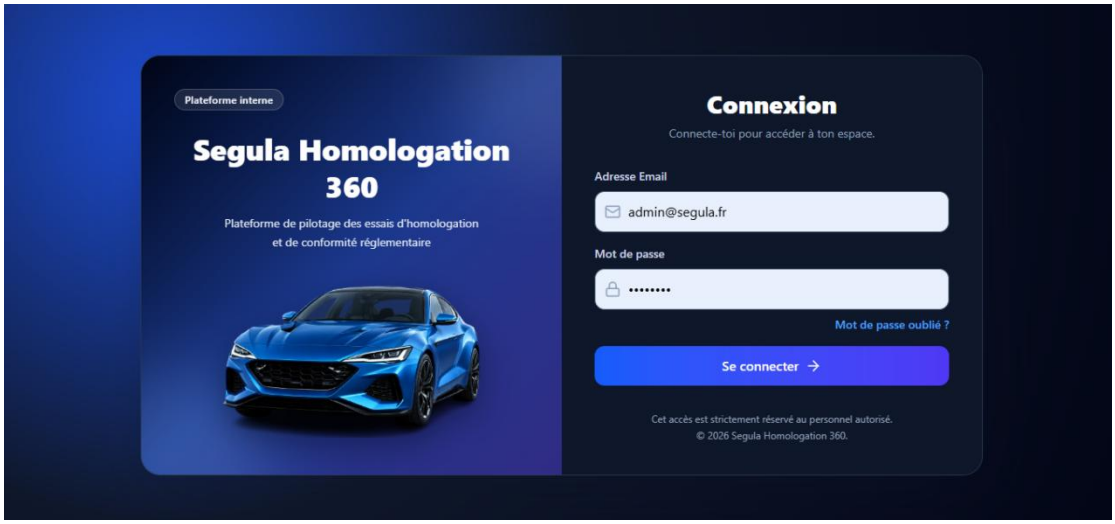
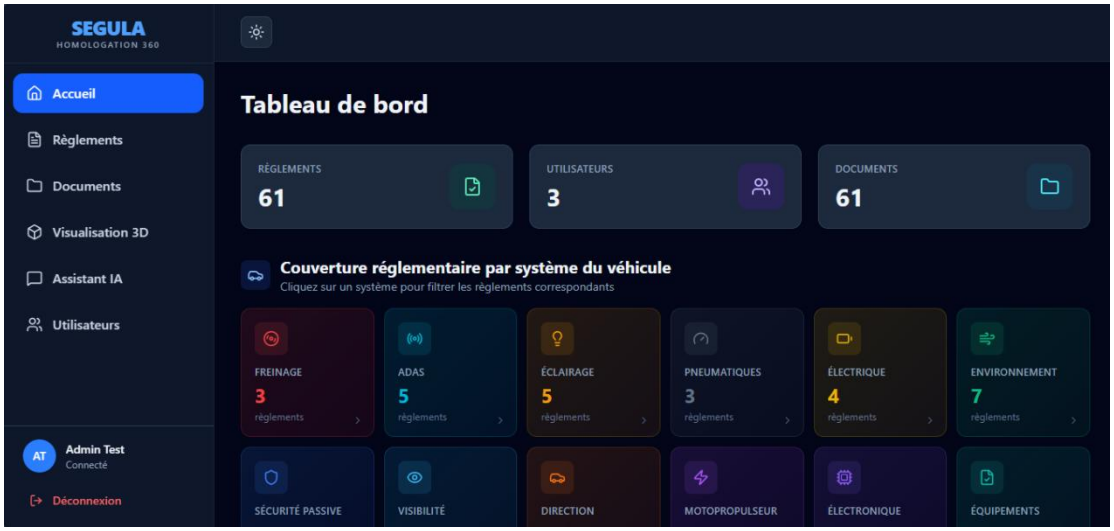


Figure 50 : Interface de connexion

4.2. Tableau de bord (Dashboard)

Le tableau de bord présente une vue globale des indicateurs clés, notamment le nombre de règlements, de documents et d'utilisateurs. Les règlements sont classés par systèmes du véhicule afin de faciliter leur consultation. Une section indique que tous les règlements proviennent de l'UNECE. Une autre met en avant les règlements disposant de supports pédagogiques. Enfin, un menu latéral permet d'accéder aux différents modules de la plateforme.



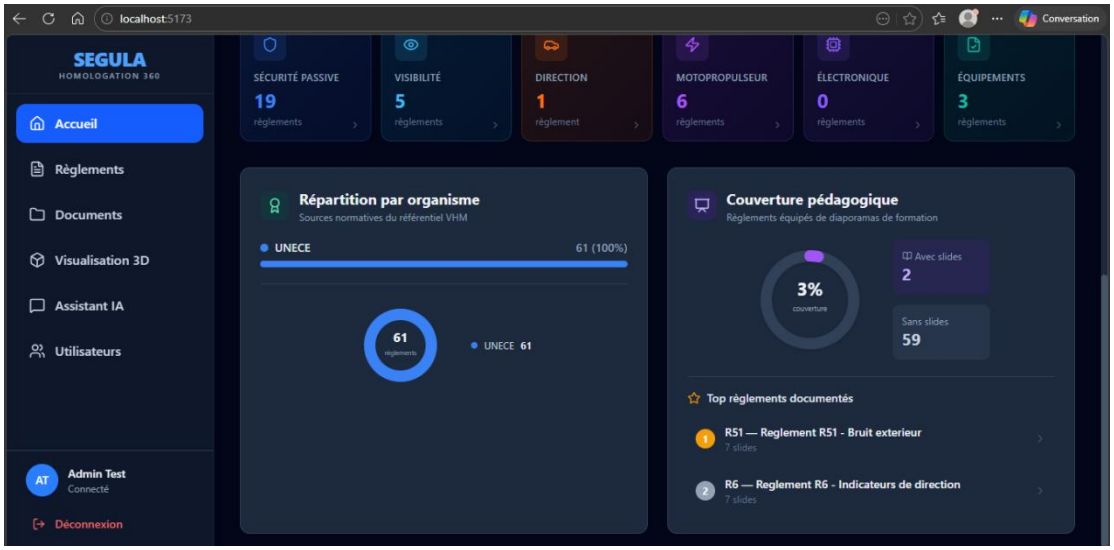


Figure 51 : Interface de Tableau de bord

4.3. Liste des règlements

Cette interface affiche l'ensemble des règlements d'homologation disponibles sous forme de cartes. L'utilisateur peut filtrer les règlements par catégorie et effectuer une recherche textuelle intelligente (par code, nom, catégorie, pièce ou mots-clés). Un tri intelligent par pertinence est également disponible. Les données sont récupérées depuis l'endpoint GET /api/v1/regulations.

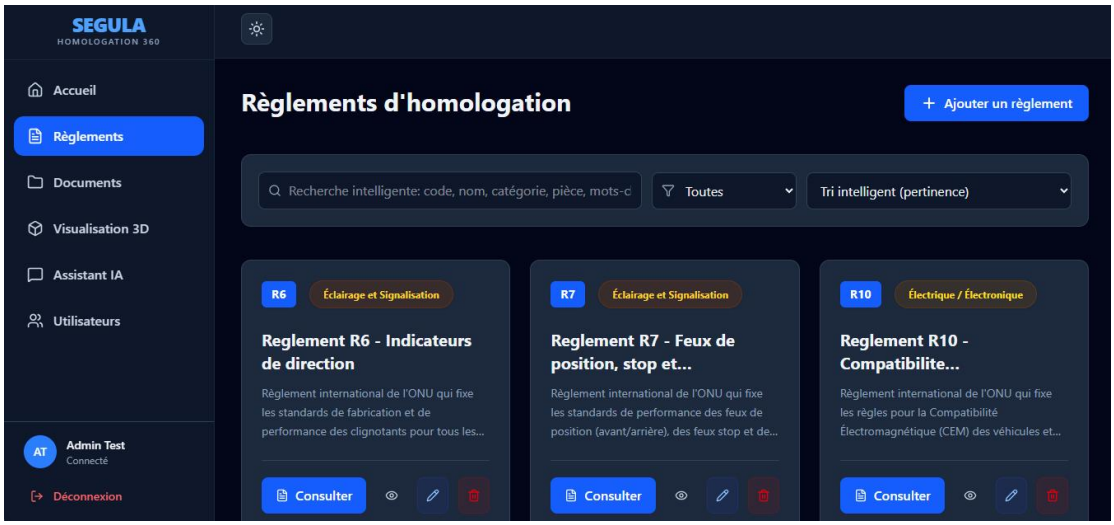
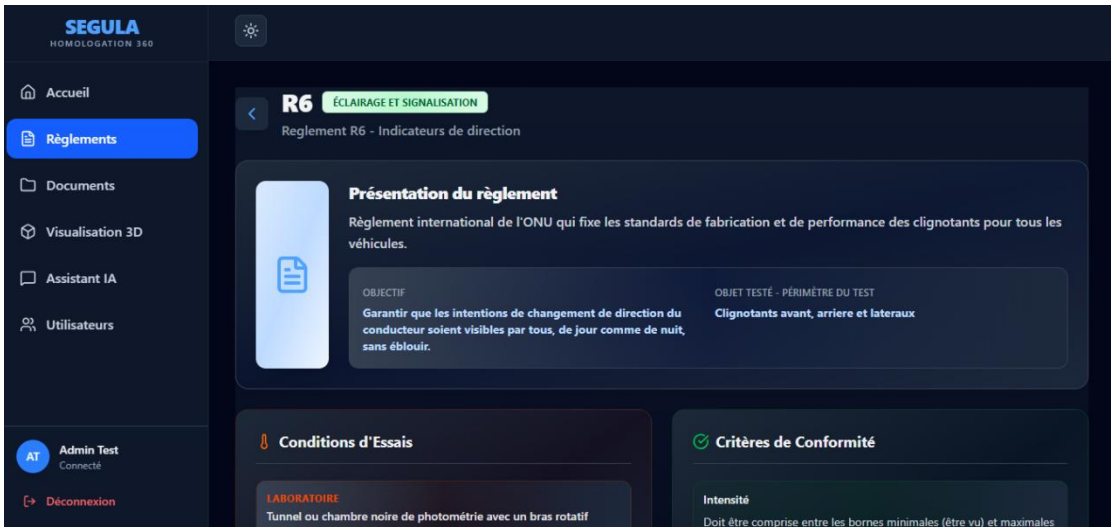
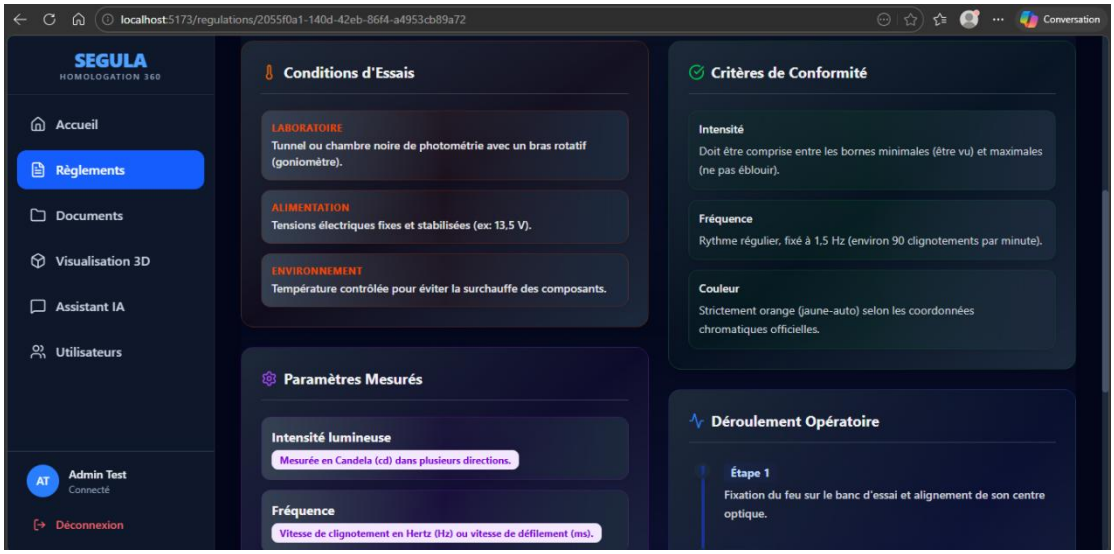


Figure 52 : Interface de liste des règlements

4.4. Détail d'un règlement

La page de détail présente l'intégralité d'un règlement sélectionné : objectif, catégorie, conditions d'essai, procédures, paramètres de mesure et critères de conformité. Ces informations sont chargées via l'endpoint GET /api/v1/regulations/{id} avec tous les champs imbriqués.





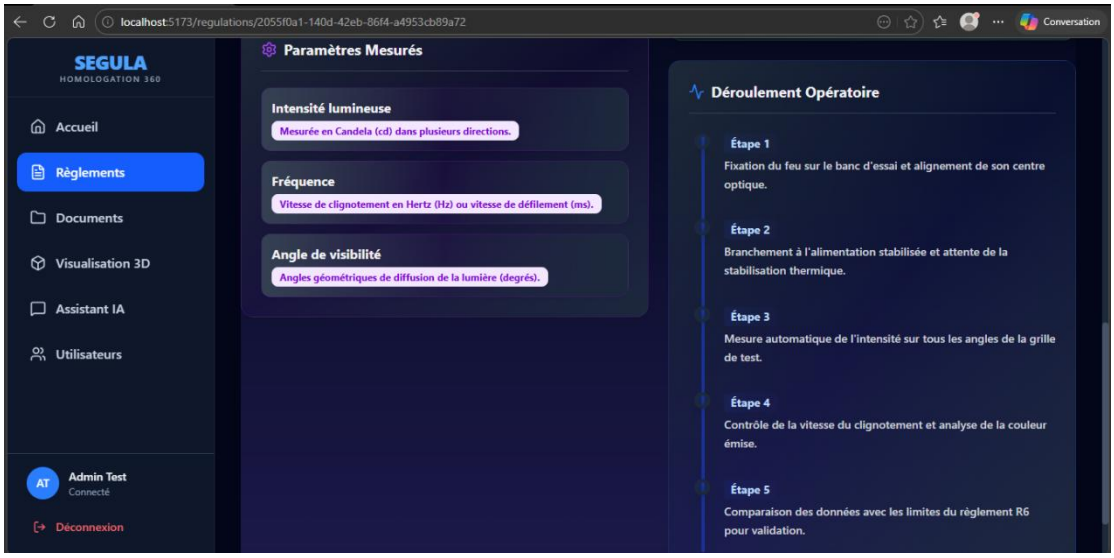


Figure 53 : Interface de détail d'un règlement

4.5. Formulaire de création / modification d'un règlement

Ce formulaire permet la création manuelle d'un nouveau règlements ou la modification d'un règlements existant. Il couvre les champs principaux (code, nom, description, objectif, catégorie) ainsi que les structures imbriquées (conditions, procédures, paramètres, critères). Les droits d'accès sont vérifiés côté backend : un ingénieur ne peut modifier que ses propres règlements, tandis que l'administrateur bénéficie d'un accès global.

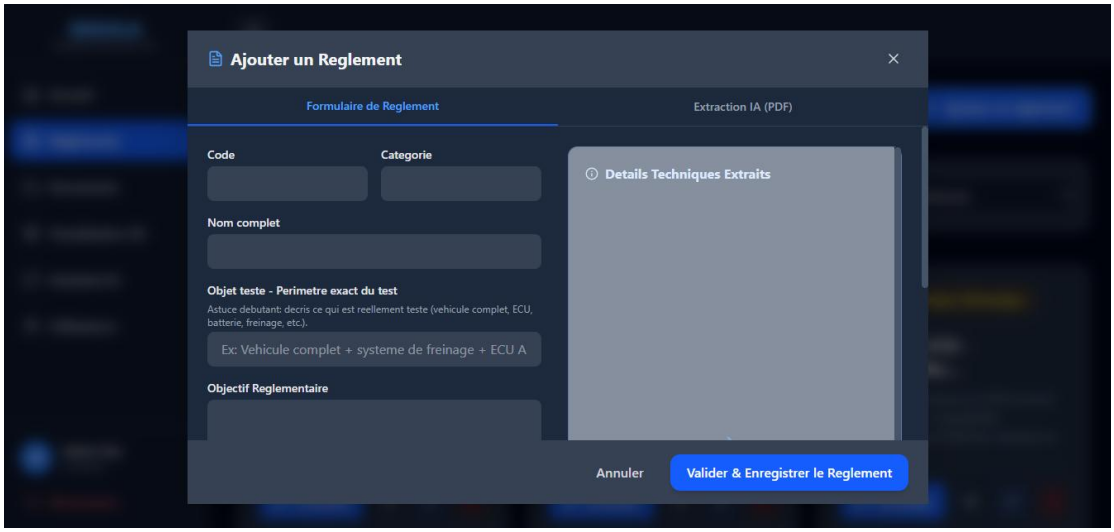


Figure 54 : Interface - Formulaire de création / modification d'un règlement

4.6. Extraction automatique depuis un PDF

Cette fonctionnalité permet à l'utilisateur d'importer un dossier réglementaire au format PDF (ex. : R51 - Règlement UNECE sur l'installation des dispositifs d'éclairage, WLTP - Procédure mondiale harmonisée d'essai). Le module d'extraction IA traite dynamiquement le document pour en extraire le contenu, les conditions métrologiques, le statut de la pièce et les critères de conformité attendus. Le backend extrait le contenu textuel via PDFBox, puis soumet ce texte à un service LLM externe qui génère une structure JSON exploitable. Le formulaire de création est ensuite automatiquement pré-rempli avec les données extraites, accélérant considérablement la saisie
réglementaire.

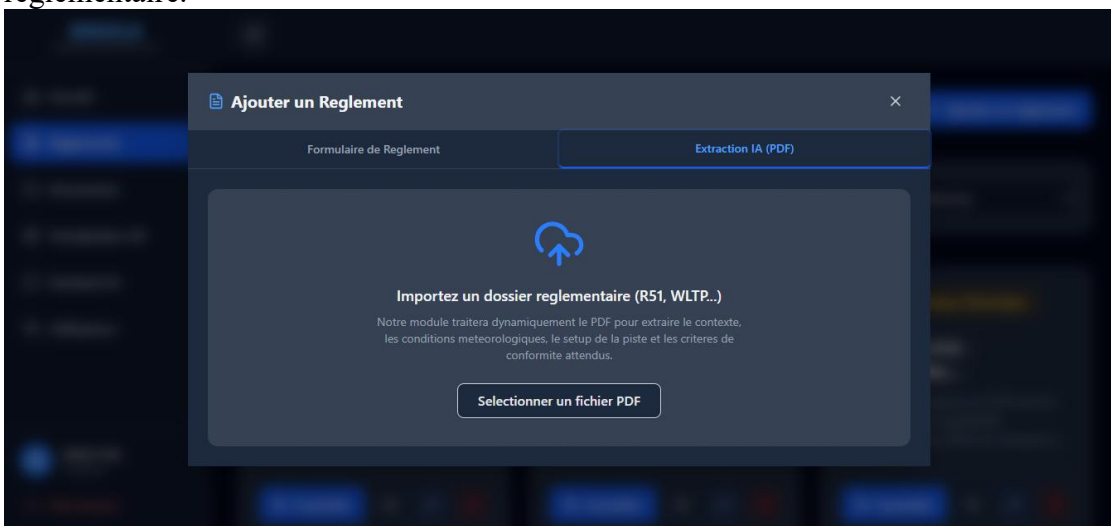


Figure 55 : Interface d'extraction automatique depuis un PDF

4.7. Support pédagogique et visualisateur de slides interactifs

Afin de faciliter la compréhension des réglementations d'homologation complexes par les ingénieurs, la plateforme intègre un module de vulgarisation et de formation sous forme de présentations interactives (slides). Ce module permet de transformer les textes réglementaires et les rapports d'essais bruts en schémas, frises chronologiques (timelines) et synthèses visuelles.

Le frontend React s'appuie sur les composants clés RegulationSlideViewer.tsx et SlideRenderer.tsx pour gérer le cycle de vie des présentations. Lorsqu'un utilisateur ouvre le visualisateur associé à une réglementation donnée, le frontend interroge l'API backend via l'endpoint GET /api/v1/regulations/{id}/slide**s**.

Cette API récupère les métadonnées ainsi que la structure JSON des slides stockées dans la base de données PostgreSQL.

Caractéristiques fonctionnelles et ergonomiques de l'interface :

- **Mode Plein Écran (Fullscreen API) :** L'utilisateur peut basculer l'affichage en plein écran natif pour masquer l'interface de l'application et projeter ou lire les slides confortablement. Une barre d'outils flottante apparaît en bas pour faciliter le contrôle.
- **Navigation Hybride :** La navigation entre les slides s'effectue soit via des boutons directionnels (Suivant / Précédent), soit directement à l'aide des touches fléchées du clavier (ArrowLeft / ArrowRight) ou de la touche d'échappement (Escape) pour fermer le visualisateur.
- **Carrousel de prévisualisation (Thumbnails) :** Un panneau situé au bas du visualisateur affiche des boutons miniatures pour chaque diapositive, indiquant le type de diapositive et son titre (**ex :** Cover, Objectifs, Timeline). Cela permet un accès direct et non-linéaire à n'importe quelle partie de la présentation.
- **Mise en page dynamique (SlideRenderer) :** En fonction du type de diapositive décrit dans le JSON, le moteur de rendu dynamique adapte l'affichage :
- **Slide de couverture (Cover) :** Affiche le code, le titre officiel, une description d'introduction et des cartes de statistiques globales.
- **Diapositive d'objectifs (Objectives) :** Présente les buts du règlement sous forme de cartes colorées utilisant des icônes issues de la bibliothèque lucide-react.
- **Timeline (Timeline / Steps) :** Affiche les étapes chronologiques des essais avec des paramètres de vitesse, de distance et des images associées (ex : trajectoires acoustiques R51, schémas de faisceaux pour le R6).
- **Formule mathématique (Formula) :** Rendu des équations réglementaires de conformité avec une légende expliquant chaque variable, symbole et unité de mesure.

- **Verdict de conformité (Conformity Visual) :** Affiche les valeurs limites réglementaires sous forme de jauges et de barres de progression horizontales, avec un bandeau de verdict visuel coloré (**Vert** pour HOMOLOGUÉ, **Rouge** pour NON-CONFORME).
- **Grille de conditions (Conditions Grid) :** Liste les exigences environnementales (piste, météo, température) sous forme de grille illustrée par des émojis.

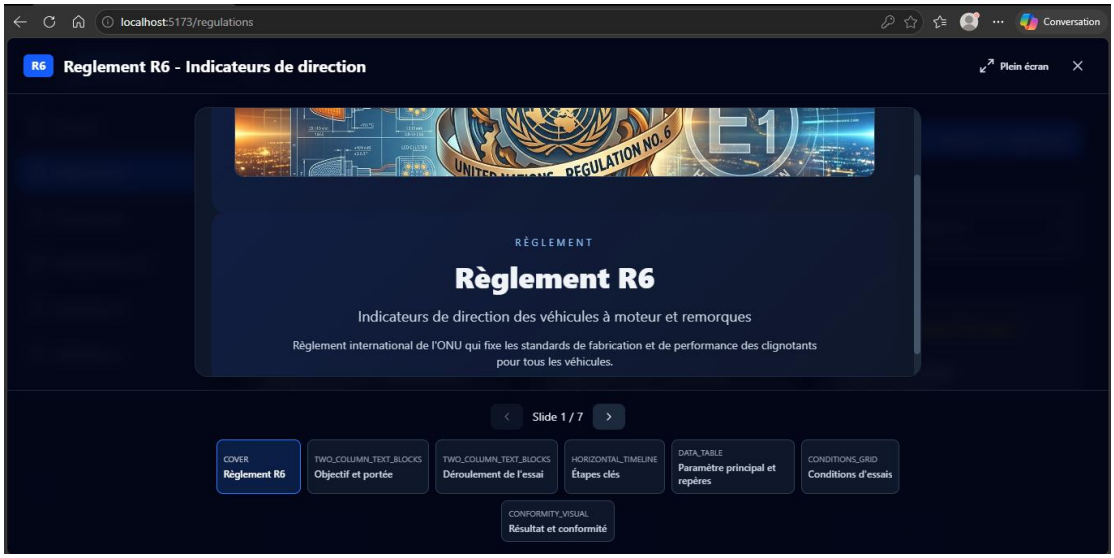


Figure 56 : Interface de visualisation des slides pédagogiques

4.8. Assistant IA (Chatbot réglementaire)

L'assistant conversationnel permet aux utilisateurs de poser des questions en langage naturel sur les règlements d'homologation. Lors du traitement d'une requête utilisateur, le backend interroge la base documentaire des règlements et sélectionne les informations les plus pertinentes afin de construire un contexte documentaire adapté. Ce contexte est ensuite transmis à un modèle de langage externe pour générer une réponse contextualisée et pertinente.

Dans l'état actuel du projet, un historique temporaire transmis par l'interface peut être utilisé afin d'améliorer la cohérence des réponses. En revanche, les échanges complets ne sont pas conservés sous forme de conversations persistantes dans la base de données. Seuls les retours utilisateurs (feed-back) peuvent être enregistrés afin de contribuer à l'amélioration continue de la qualité des réponses fournies par l'assistant.

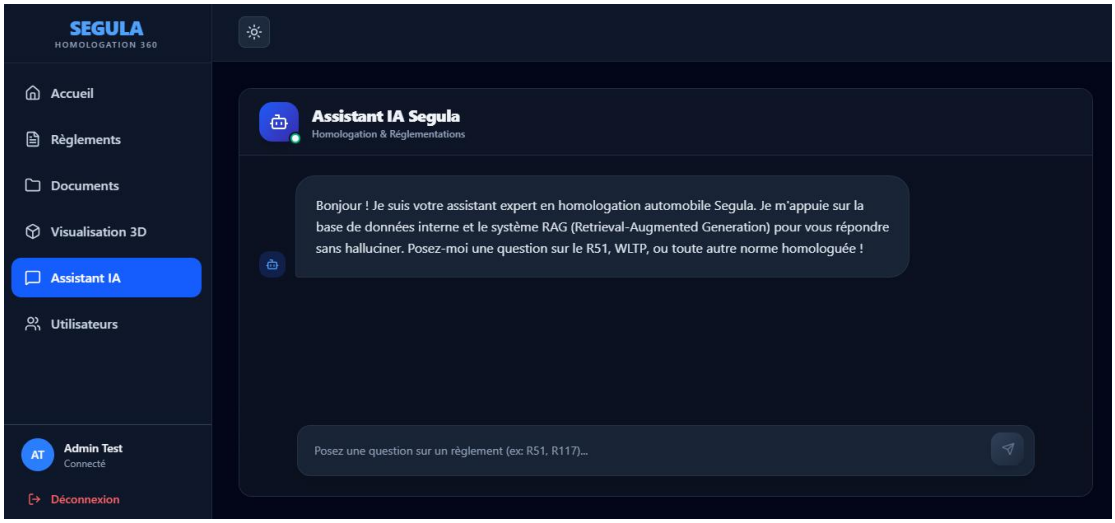


Figure 57 : Interface - Assistant IA

4.8.1. Architecture du module IA

L'assistant conversationnel, nommé « Assistant IA Segula – Homologation & Réglementations », permet aux utilisateurs de poser des questions en langage naturel sur les règlements d'homologation. Il s'appuie sur la base de données interne et le système RAG (Retrieval-Augmented Generation) pour fournir des réponses contextualisées et fiables.

Lors du traitement d'une requête utilisateur, le backend interroge la base documentaire des règlements et récupère les informations réglementaires pertinentes afin de construire un contexte documentaire adapté. Ce contexte est ensuite transmis au modèle de langage Llama 3 pour générer une réponse contextualisée.

Dans l'état actuel du projet, un historique temporaire transmis par l'interface peut être utilisé afin d'améliorer la cohérence des réponses. En revanche, les échanges complets ne sont pas conservés sous forme de conversations persistantes dans la base de données. Seuls les retours utilisateurs (feed-back) peuvent être enregistrés via le composant ChatFeedback afin de contribuer à l'amélioration continue de la qualité des réponses fournies par l'assistant.

4.8.2. Pipeline RAG

Afin d'améliorer la pertinence des réponses générées, l'assistant intelligent repose sur une approche de type Retrieval-Augmented Generation (RAG). Cette architecture permet d'enrichir les requêtes utilisateur à partir des informations réglementaires déjà stockées dans la base documentaire de la plateforme avant leur transmission au modèle de langage Llama 3.

Le fonctionnement du pipeline RAG se déroule selon les étapes suivantes :

- Question utilisateur : l'utilisateur pose une question en langage naturel depuis l'interface conversationnelle.

- Récupération des données réglementaires : le backend interroge la base documentaire (RegulationRepository / findAll()) et récupère les règlements pertinents.
- Construction du contexte : les informations récupérées sont regroupées par le ContextBuilder afin de construire un contexte documentaire adapté.
- Envoi au modèle de langage : le contexte et la question utilisateur sont transmis au modèle de langage externe Llama 3.
- Génération de la réponse : le modèle génère une réponse contextualisée à partir du contexte fourni.
- Retour à l'utilisateur : la réponse générée est affichée dans l'interface conversationnelle.

Deux mécanismes optionnels complètent ce pipeline :

- **Historique temporaire :** l'interface peut envoyer un historique temporaire pour améliorer la cohérence des réponses. Cet historique n'est pas conservé en base.
- **Enregistrement du feedback :** après la réponse, l'utilisateur peut donner un retour. Seuls les feedbacks sont enregistrés en base (via ChatFeedback) pour améliorer la qualité des réponses.

L'extraction et la structuration des documents PDF réglementaires sont réalisées en amont lors de l'importation des règlements dans la plateforme. Ces opérations ne font donc pas partie du processus de génération des réponses de l'assistant conversationnel.

Cette approche permet de réduire les risques d'hallucination du modèle et d'améliorer la qualité des réponses grâce à l'exploitation des données réglementaires intégrées dans la plateforme.

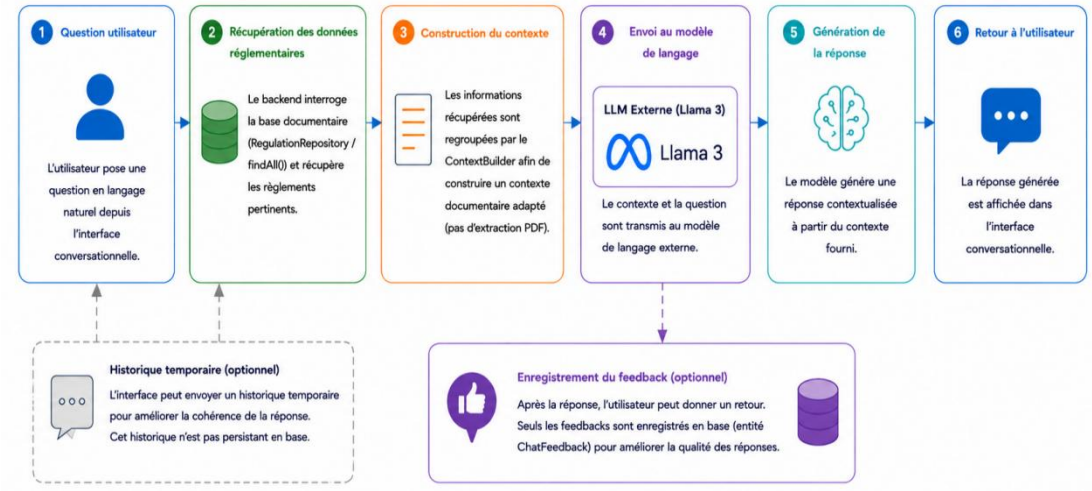


Figure 58 : Pipeline RAG

4.9. Visualisation 3D du véhicule

Le module de visualisation 3D, développé en Three.js directement dans le frontend, charge un modèle de véhicule au format GLB dans un environnement de simulation 3D interactif.

Sélection du type de motorisation

L'utilisateur sélectionne le type de motorisation souhaité parmi trois options : Thermique (ICE), Hybride ou Électrique (EV). Chaque type de motorisation est associé à un modèle de véhicule distinct, affiché automatiquement dans l'environnement 3D lors de la sélection. Par défaut, le type Thermique est présélectionné au chargement de la page. Ce choix conditionne à la fois le modèle 3D affiché et l'ensemble des règlements applicables.

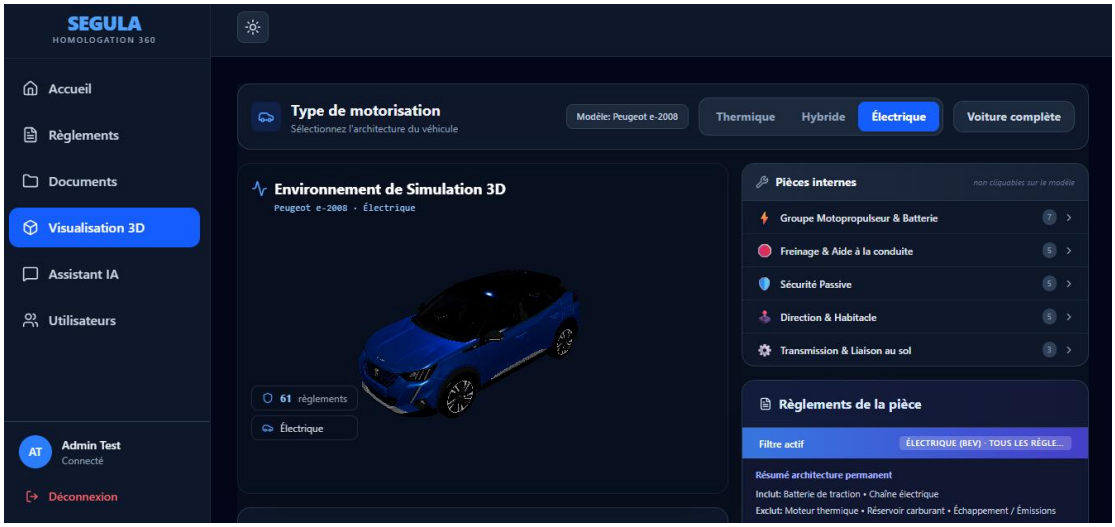
Interaction avec les pièces du véhicule

Le module distingue deux catégories de pièces :

- **Pièces externes :** directement cliquables sur le modèle 3D, elles permettent à l'utilisateur de sélectionner visuellement une zone du véhicule telle que les vitres latérales, le pare-brise, les rétroviseurs, les projecteurs, les indicateurs de direction, les pare-chocs, la plaque d'immatriculation, les feux, le pot d'échappement, les barres de toit ou l'antenne.
- **Pièces internes :** non représentées de manière cliquable sur le modèle 3D, elles sont accessibles depuis le panneau latéral. Elles regroupent les catégories suivantes : Groupe Motopropulseur & Batterie, Freinage & Aide à la conduite, Sécurité Passive, Direction & Habitacle, et Transmission & Liaison au sol.

Filtrage dynamique des règlements

Lorsqu'une pièce est sélectionnée, les règlements d'homologation applicables sont automatiquement filtrés et affichés dans le panneau « Règlements de la pièce ». Un système de filtres actifs, matérialisé par des tags, indique le type de motorisation ainsi que la pièce sélectionnée. L'utilisateur peut à tout moment réinitialiser l'ensemble des filtres afin de revenir à la vue globale des règlements.



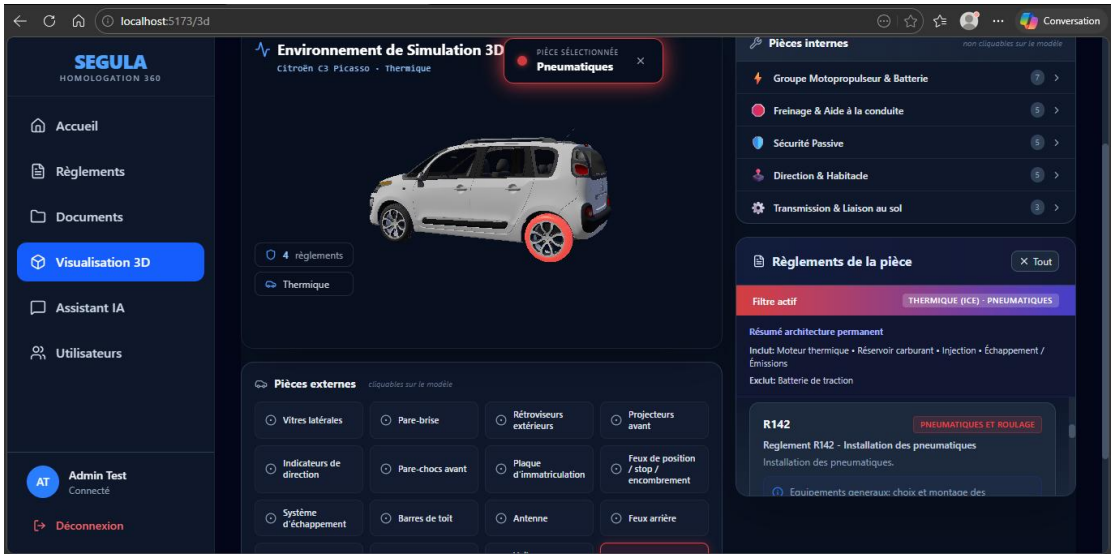


Figure 59 : Interface - Visualisation 3D du véhicule

4.10. Administration des utilisateurs

Accessible uniquement aux administrateurs, cette interface permet de gérer les comptes utilisateurs : création, modification des informations de profil et du rôle, et suppression. Les opérations sont réalisées via les endpoints REST GET/POST/PUT/DELETE /api/v1/users.

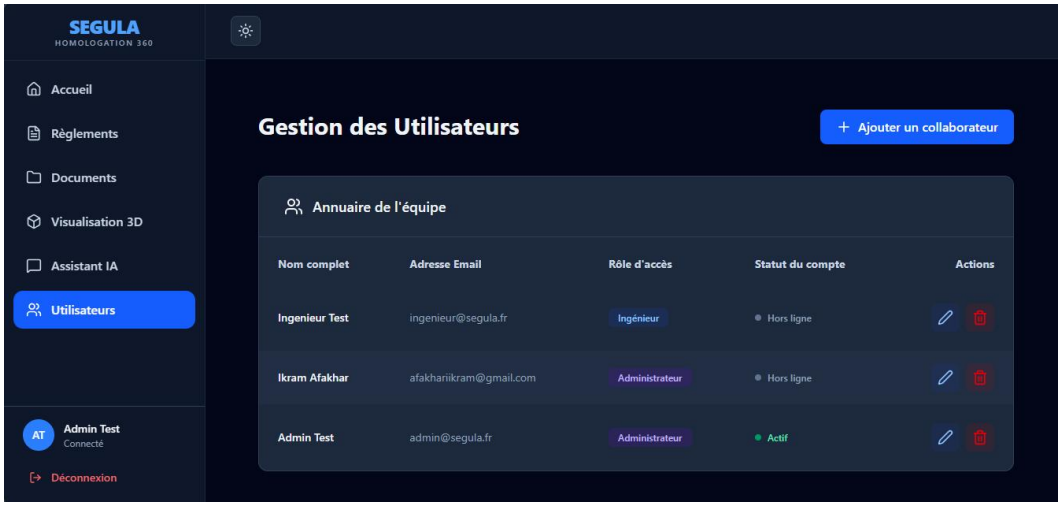


Figure 60 : Interface de gestion des utilisateurs

4.11. Catalogue de documents réglementaires

Cette interface donne accès à la bibliothèque documentaire UNECE intégrée à la plateforme. Les documents sont classés par catégorie et filtrables par thématique. L'utilisateur peut consulter les liens PDF officiels directement depuis l'application.

Les données sont alimentées par un module de scraping Python collectant automatiquement les métadonnées et URLs réglementaires.

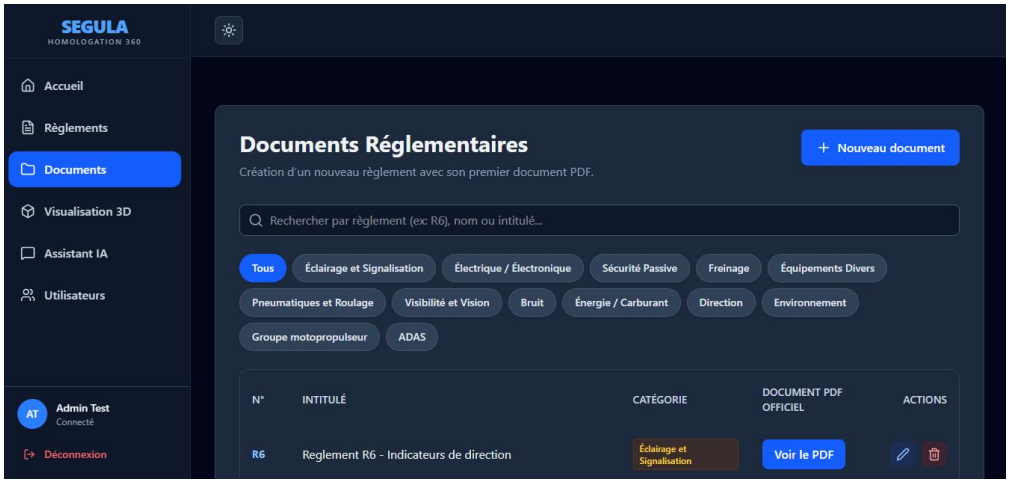


Figure 61 : Interface de catalogue de documents réglementaires

5. Intégration et déploiement

L'application est structurée pour un déploiement flexible aussi bien en environnement local que de production. La conteneurisation via Docker Compose unifie les trois services (frontend, backend, base de données) dans un environnement d'exécution cohérent et portable.

5.1. Architecture de déploiement local

En développement, l'orchestration est assurée par Docker Compose. Le frontend React est servi via Nginx qui joue également le rôle de proxy inverse vers le backend Spring Boot. La base de données PostgreSQL est isolée dans son propre conteneur avec un volume persistant.

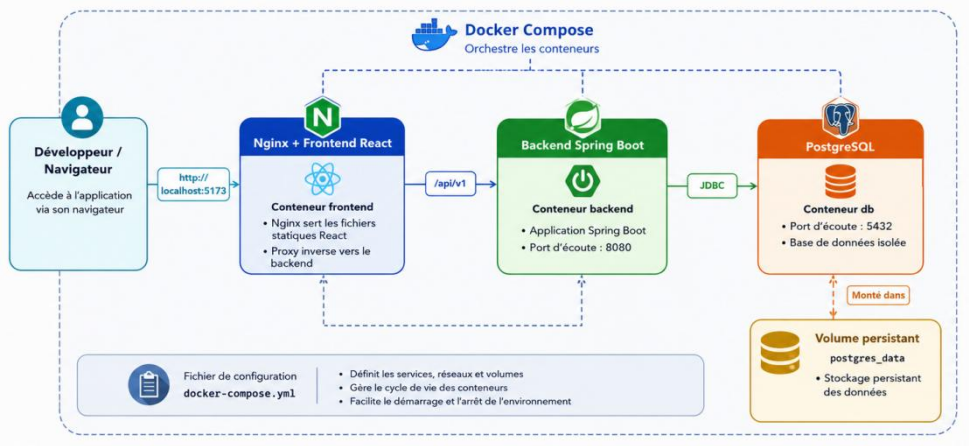


Figure 62 : Architecture technique de la solution

5.2. Pipeline CI avec GitHub Actions

Un pipeline d'intégration continue est mis en place via GitHub Actions. À chaque push sur les branches principales, le pipeline déclenche automatiquement : la vérification de la syntaxe et du typage TypeScript (frontend), la compilation Maven et les tests unitaires (backend), et la construction des images Docker. Ce pipeline garantit que chaque version du code est valide avant toute tentative de déploiement.

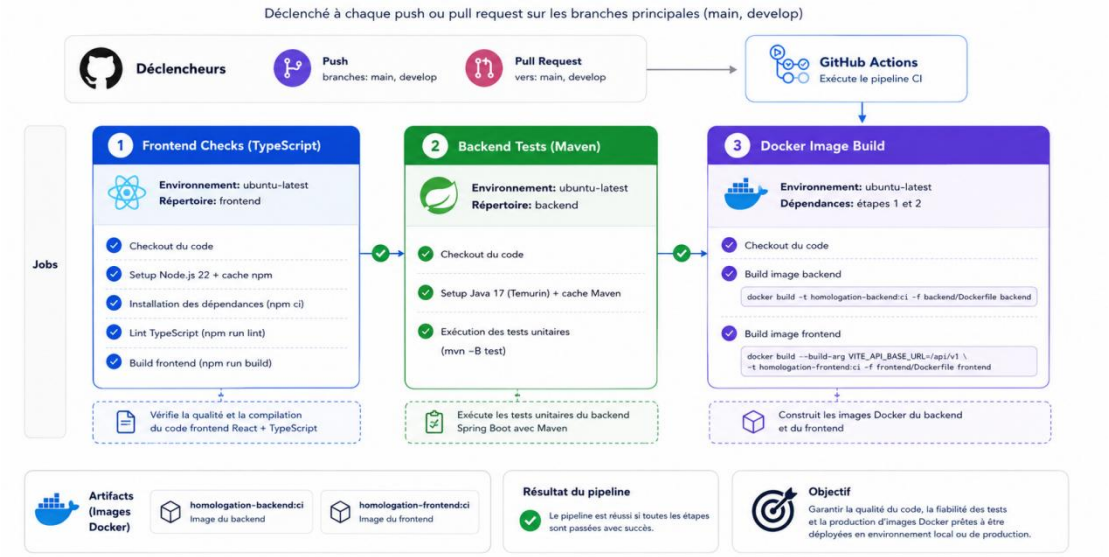


Figure 63 : Pipeline CI avec GitHub Actions

5.3. Stratégie de tests et automatisation

Pour garantir la maintenabilité, la non-régression et la robustesse de la plateforme « Segula Homologation 360 », une stratégie de tests multiniveaux a été mise en œuvre, couvrant le backend, le frontend et la validation globale de navigation.

5.3.1. Tests unitaires (Backend Java)

Les tests unitaires du backend s'appuient sur les frameworks JUnit 5 et Mockito pour isoler la logique métier des dépendances d'infrastructure (base de données, services tiers).

La classe RegulationServiceTest.java illustre cette approche :

- **Simulation des Repositories :** L'usage de @Mock pour RegulationRepository permet de simuler les opérations de sauvegarde et de recherche en base de données, évitant de modifier la base réelle lors des phases de test.
- **Validation de la logique métier et de normalisation :** Les cas de test vérifient que le service applique correctement les règles métiers de Segula :
  - Normalisation du code du règlement (ex : conversion automatique en majuscules comme "r51" en "R51").

- Déduction automatique de la catégorie (ex : un règlement avec le code "R154" est automatiquement classé dans la catégorie "Environnement").
- Pré-remplissage des champs de test (objetTest et organismeOrigine).
- Mappings corrects des entités imbriquées (conditions, procédures, paramètres de mesure, critères de conformité et diapositives).

5.3.2. Tests d'intégration et navigation automatisée (E2E Frontend)

Les tests de parcours utilisateur et de rendu graphique sont automatisés à l'aide d'un script écrit en Puppeteer (test-nav.cjs). Ce script simule les actions d'un utilisateur dans un navigateur Chromium sans tête (headless) :

- **Authentification automatique :** Il navigue sur la page de connexion, remplit les champs de saisie (e-mail et mot de passe de l'administrateur admin@segula.fr) et soumet le formulaire.
- **Vérification du tableau de bord :** Il valide la transition après connexion et l'obtention du token JWT.
- **Validation 3D :** Il navigue sur l'URL de simulation 3D (/3d), attend l'apparition de l'élément HTML <canvas> (moteur de rendu WebGL de Three.js) pour s'assurer que le modèle 3D du véhicule est correctement chargé et affiché à l'écran.
- **Navigation générale :** Il teste le bon fonctionnement des liens de navigation (ex. : clic sur le lien "Accueil" pour retourner au tableau de bord) et surveille les erreurs console ou JavaScript levées par le navigateur.

5.3.3. Analyse statique et qualité de code

- **Frontend :** L'outil ESLint applique des règles de formatage et de détection d'erreurs potentielles (**ex :** respect des règles de hooks React, variables inutilisées). Le compilateur TypeScript (tsc) valide statiquement le typage des props, des états et des appels API pour éviter les erreurs de type à l'exécution.
- **Backend :** La compilation stricte via Java 17 et Maven garantit l'intégrité du code source.

5.3.4. Automatisation dans le pipeline CI (GitHub Actions)

L'ensemble de ces contrôles et tests est intégré et automatisé au sein d'un pipeline d'Intégration Continue (CI) via GitHub Actions (ci.yml).

À chaque validation de code (push ou pull_request sur les branches main et develop), le serveur de CI exécute automatiquement :

- **Vérifications Frontend :** Restauration des dépendances avec npm ci, linting statique avec npm run lint et build de production pour valider la compilation globale.
- **Tests Backend :** Compilation du projet et exécution de toute la suite de tests JUnit via la commande Maven mvn -B test contrôlée par le plugin Surefire.

- **Validation Docker :** Construction des images Docker du frontend et du backend (via les fichiers Dockerfile respectifs) pour valider que l'application reste packagée et déployable sans erreur.

6. Conclusion

La phase de réalisation a permis de transformer la conception initiale en une solution opérationnelle, organisée autour d'une architecture claire et évolutive. Les interfaces développées couvrent l'ensemble des fonctionnalités définies lors de l'analyse des besoins : gestion des règlements, suivi des sessions d'essai, assistance IA, visualisation 3D et administration. La conteneurisation et le pipeline CI constituent une base solide pour les évolutions futures du projet.

Chapitre 5

Résultats, limites et perspectives

1. Introduction

Ce chapitre présente un bilan global du projet « Segula Homologation 360 » après sa phase de conception et de développement. Il met en évidence les résultats obtenus à travers les différentes fonctionnalités implémentées ainsi que les bénéfices apportés au processus d'homologation automobile.

Ce chapitre analyse également les limites identifiées au cours de la réalisation du projet, qu'elles soient techniques, fonctionnelles ou organisationnelles. Enfin, il propose plusieurs perspectives d'évolution permettant d'améliorer la solution et de la faire évoluer vers une plateforme industrielle plus robuste, sécurisée et évolutive.

2. Résultats obtenus

Le projet a permis de concevoir et de développer une plateforme web centralisée destinée à faciliter la gestion des règlements d'homologation automobile. La solution développée répond aux principales problématiques identifiées lors de l'étude des besoins, notamment la centralisation des informations réglementaires, l'amélioration de leur accessibilité et l'automatisation de certaines tâches complexes.

2.1. Fonctionnalités implémentées

La plateforme intègre plusieurs fonctionnalités majeures :

2.1.1. Authentification et gestion des rôles

Un système sécurisé d'authentification a été mis en place afin de contrôler l'accès aux différentes fonctionnalités de la plateforme. Les utilisateurs sont répartis selon plusieurs rôles, permettant une gestion fine des autorisations et des responsabilités.

2.1.2. Gestion des règlements

La plateforme permet la création, la consultation, la modification et la suppression des règlements d'homologation. Chaque règlement peut être associé à différents éléments métier tels que :

- Les conditions d'essai
- Les procédures d'essai
- Les paramètres mesurés
- Les critères de conformité
- Les documents réglementaires associés

Cette organisation permet une structuration claire des informations et facilite leur exploitation.

2.1.3. Extraction intelligente des données réglementaires

L'une des principales innovations du projet réside dans l'intégration d'un mécanisme d'extraction automatique des informations à partir de documents PDF réglementaires.

Grâce à l'utilisation de techniques d'intelligence artificielle, les documents sont analysés puis transformés en données structurées directement exploitables par la plateforme. Cette automatisation réduit considérablement le temps nécessaire à la saisie manuelle des informations.

2.1.4. Assistant conversationnel basé sur l'intelligence artificielle

Un assistant intelligent a été intégré afin de permettre aux utilisateurs d'interroger la base réglementaire en langage naturel.

Cette fonctionnalité facilite la recherche d'informations dans les règlements et améliore l'accessibilité des connaissances réglementaires, notamment pour les utilisateurs non experts.

2.1.5. Visualisation 3D des véhicules

La plateforme propose une visualisation tridimensionnelle interactive permettant d'explorer différentes parties du véhicule.

Cette fonctionnalité améliore la compréhension des composants concernés par les règlements et offre une expérience utilisateur moderne et intuitive.

2.1.6. Tableau de bord décisionnel

Un tableau de bord centralise les principaux indicateurs relatifs aux règlements, aux documents et à l'activité des utilisateurs.

Il fournit une vue synthétique permettant d'améliorer le suivi des données et d'accélérer la prise de décision.

2.1.7. Administration de la plateforme

Des outils d'administration permettent la gestion :

- des utilisateurs
- des rôles
- des règlements
- des documents réglementaires

Cette fonctionnalité garantit une gestion centralisée et sécurisée de l'ensemble du système.

2.2. Bénéfices apportés par la solution

La plateforme développée apporte plusieurs améliorations significatives au processus d'homologation automobile.

Tout d'abord, elle permet la centralisation de l'ensemble des informations réglementaires dans un environnement unique, réduisant ainsi la dispersion des données entre différents supports.

L'automatisation de l'extraction documentaire réduit considérablement les tâches manuelles et limite les risques d'erreurs lors de la saisie des informations.

L'assistant intelligent améliore l'accès à la connaissance réglementaire grâce à des recherches rapides en langage naturel.

La visualisation 3D offre une approche plus intuitive de la réglementation en facilitant l'identification des composants concernés par les différents essais d'homologation.

Enfin, l'ensemble de ces fonctionnalités contribue à améliorer la productivité des ingénieurs et à accélérer l'analyse des exigences réglementaires.

2.3. Validation des besoins identifiés

L'analyse des résultats montre que la majorité des besoins fonctionnels définis dans le cahier des charges ont été satisfaits.

Les fonctionnalités principales identifiées lors de la phase d'analyse ont été réalisées avec succès :

Besoin identifié	Réalisation
Centralisation des règlements	✓
Gestion documentaire	✓
Extraction automatique PDF	✓
Assistant intelligent	✓
Gestion des utilisateurs	✓

Table 3 : Validation des besoins identifiés

La solution développée constitue ainsi une réponse concrète aux problématiques rencontrées par les équipes d'homologation.

3. Limites de la solution

Malgré les résultats obtenus, certaines limites subsistent dans la version actuelle de la plateforme.

3.1. Dépendance aux services d'intelligence artificielle

Les fonctionnalités d'extraction documentaire et d'assistance conversationnelle reposent sur des services d'intelligence artificielle externes.

Cette dépendance introduit plusieurs contraintes :

- nécessité d'une connexion réseau ;
- dépendance à la disponibilité du fournisseur ;
- éventuels coûts d'utilisation ;
- latence variable selon les performances du service.

3.2. Limites du mécanisme de recherche intelligent

Le système actuel repose principalement sur une recherche basée sur les données stockées dans la base relationnelle.

Bien que cette approche soit adaptée à la taille actuelle de la plateforme, elle présente certaines limites lorsque le volume documentaire devient très important.

Une recherche sémantique plus avancée permettrait d'améliorer davantage la pertinence des réponses générées par l'assistant.

3.3. Gestion des évolutions réglementaires

Les règlements automobiles évoluent régulièrement à travers de nouvelles versions et amendements.

La plateforme ne gère pas encore de manière complète l’historisation et le versioning des règlements, ce qui peut compliquer le suivi des évolutions réglementaires dans le temps.

3.4. Sécurité et gestion des secrets

Bien que plusieurs mécanismes de sécurité aient été mis en œuvre, certains aspects peuvent encore être améliorés :

- gestion centralisée des secrets
- rotation des clés d'accès
- supervision des accès
- audit de sécurité complet

3.5. Limites liées au contexte du projet

Le projet a été réalisé dans un contexte académique avec des contraintes de temps et de ressources.

Certaines fonctionnalités avancées prévues lors de la phase de conception n'ont donc pas pu être implémentées dans cette première version.

4. Perspectives d'évolution

Afin d'améliorer la plateforme et de renforcer sa valeur ajoutée pour les équipes d'homologation, plusieurs axes d'évolution peuvent être envisagés.

4.1. Amélioration du moteur d'intelligence artificielle

Les futures versions pourront intégrer :

- des modèles plus spécialisés dans le domaine réglementaire
- une meilleure compréhension du contexte des requêtes
- des réponses plus précises et documentées
- des mécanismes de citation automatique des sources

4.2. Mise en place d'un système RAG avancé

L'intégration d'une architecture RAG (Retrieval-Augmented Generation) plus avancée permettrait :

- l'indexation sémantique des règlements
- la recherche vectorielle
- l'amélioration de la pertinence des réponses
- la réduction des hallucinations des modèles d'IA

4.3. Versioning des règlements

Un système de gestion des versions pourrait être ajouté afin :

- de conserver l'historique des règlements
- de suivre les modifications réglementaires
- de faciliter les audits de conformité
- d'améliorer la traçabilité des essais réalisés

4.4. Tableaux de bord avancés

Des tableaux de bord plus complets pourraient être développés afin de fournir :

- Des statistiques détaillées
- Des indicateurs de conformité
- Des analyses comparatives
- Des outils d'aide à la décision

4.5. Renforcement de la sécurité

Les évolutions futures pourront inclure :

- L'authentification multifactorielle
- Le chiffrement avancé des données
- La gestion centralisée des secrets
- L'audit des activités utilisateurs

4.6. Évolution vers le Déploiement Continu (CD) et l'hébergement Cloud

À ce jour, le pipeline DevOps du projet prend en charge l'Intégration Continue (CI) automatisée grâce à GitHub Actions ainsi que la conteneurisation via Docker.

L'une des principales perspectives consiste à mettre en place une chaîne complète de Déploiement Continu (CD) vers un environnement Cloud tel qu'AWS, Azure ou Google Cloud Platform.

Cette évolution permettra d'automatiser l'ensemble du cycle de déploiement, depuis la validation du code jusqu'à sa mise en production.

Les principales étapes envisagées sont :

Automatisation du pipeline CD

Les workflows GitHub Actions seront étendus afin de :

- construire automatiquement les images Docker
- exécuter les tests automatisés
- effectuer des contrôles de sécurité
- publier les images dans un registre privé

Déploiement avec Kubernetes

L'application pourra être déployée sur un cluster Kubernetes afin de bénéficier :

- D'une haute disponibilité
- D'une meilleure gestion des ressources
- D'une montée en charge automatique
- D'une gestion centralisée des configurations et secrets

Supervision et observabilité

L'intégration de solutions telles que Prometheus et Grafana permettra :

- La collecte de métriques
- La surveillance des performances
- La détection précoce des anomalies
- L'analyse de la disponibilité des services

Déploiement sans interruption

Des stratégies telles que :

- Rolling Update
- Blue-Green Deployment

permettront de déployer les nouvelles versions sans interruption de service pour les utilisateurs.

5. Analyse critique

5.1. Difficultés rencontrées

La réalisation du projet a présenté plusieurs défis techniques, notamment :

- l'intégration de services d'intelligence artificielle
- l'extraction automatique d'informations depuis des documents complexes
- la modélisation des données réglementaires
- la gestion de la visualisation 3D dans une application web

Ces difficultés ont nécessité plusieurs phases d'expérimentation et d'optimisation.

5.2. Pertinence des choix technologiques

Les technologies retenues se sont révélées adaptées aux objectifs du projet.

- **React et TypeScript** ont permis de développer une interface moderne et maintenable.
- **Spring Boot** a facilité la mise en place d'une architecture robuste et sécurisée.
- **PostgreSQL** a assuré une gestion fiable des données.
- **Docker** a simplifié le déploiement et la reproductibilité des environnements.
- **GitHub Actions** a permis d'automatiser les processus d'intégration continue.

5.3. Compétences acquises

Ce projet a permis de renforcer plusieurs compétences :

- Développement Full Stack.
- Architecture logicielle.
- Bases de données relationnelles.
- Intelligence artificielle générative.
- DevOps et intégration continue.
- Sécurité applicative.
- Gestion de projet.

6. Conclusion

La plateforme « Segula Homologation 360 » constitue une solution innovante répondant aux principaux besoins liés à la gestion des règlements d'homologation automobile.

Les fonctionnalités développées permettent de centraliser les données réglementaires, d'automatiser certaines tâches complexes et d'améliorer l'accès à l'information grâce à l'intelligence artificielle.

Malgré quelques limitations inhérentes à cette première version, la solution offre une base solide et évolutive. Les perspectives identifiées, notamment l'intégration d'un RAG avancé, le déploiement Cloud et l'orchestration Kubernetes, ouvrent la voie vers une plateforme industrielle capable d'accompagner durablement les activités d'homologation automobile.

Conclusion générale

Ce projet a permis de concevoir et de réaliser une plateforme complète dédiée à la gestion et à l'analyse des règlements d'homologation automobile, baptisée « Segula Homologation 360 ». L'objectif principal était de centraliser les données réglementaires, d'automatiser leur traitement et d'améliorer l'accès à l'information grâce à des technologies modernes.

L'architecture adoptée, basée sur un modèle 3-tiers (Frontend, Backend, Base de données), a garanti une bonne séparation des responsabilités, tout en assurant la scalabilité et la maintenabilité du système. L'intégration de technologies telles que React, Spring Boot et PostgreSQL a permis de construire une application robuste et performante.

L'apport majeur de ce projet réside dans l'intégration de modules avancés, notamment l'intelligence artificielle (LLM et RAG) pour l'assistance intelligente, ainsi que la visualisation 3D interactive. Ces fonctionnalités innovantes contribuent à améliorer significativement l'expérience utilisateur et la compréhension des données techniques.

Cependant, certaines limites subsistent, notamment la dépendance à des services externes pour les modèles d'intelligence artificielle et l'absence de déploiement complet sur une infrastructure cloud avec Kubernetes. Ces aspects constituent des perspectives d'amélioration pour de futures évolutions.

En conclusion, ce projet représente une contribution significative dans le domaine de la digitalisation des processus d'homologation, en proposant une solution moderne, évolutive et intelligente, répondant aux besoins actuels de l'industrie automobile.

Webographie Et Bibliographie

1. Ouvrages, Articles Académiques et Normes (Bibliographie)

[1] Lewis, P. et al. – "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". NeurIPS, 2020. (Référence pour l'architecture IA / RAG).

[2] IETF – JSON Web Token (JWT) - RFC 7519. Disponible sur : jwt.io

[3] OMG – Unified Modeling Language (UML) Specification. Disponible sur : omg.org/spec/UML/

[4] Sommerville, I. – Software Engineering. Pearson, 2016.

[5] Fowler, M. – Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.

[6] Gamma, E. et al. – Design Patterns. Addison-Wesley, 1994.

[7] UNECE – Règlements concernant l'homologation des véhicules. Disponible sur : unece.org/transport/vehicle-regulations

2. Documentations Techniques Officielles (Webographie)

[8] Front-end et rendu 3D : React (react.dev), Three.js et React Three Fiber (threejs.org), Tailwind CSS (tailwindcss.com), TypeScript (typescriptlang.org), Vite et Axios.

[9] Back-end, base de données et traitement : Spring Boot (spring.io), PostgreSQL (postgresql.org), Hibernate/JPA, Apache PDFBox (pdfbox.apache.org) et Flyway.

[10] Intelligence artificielle : modèle Llama 3 de Meta (llama.meta.com), servi via Groq API (console.groq.com).

[11] DevOps et supervision : Docker (docker.com), Kubernetes (kubernetes.io), GitHub Actions (docs.github.com), Grafana (grafana.com) et Prometheus (prometheus.io).